

Quantization & Pruning



Computation Reduction Approaches

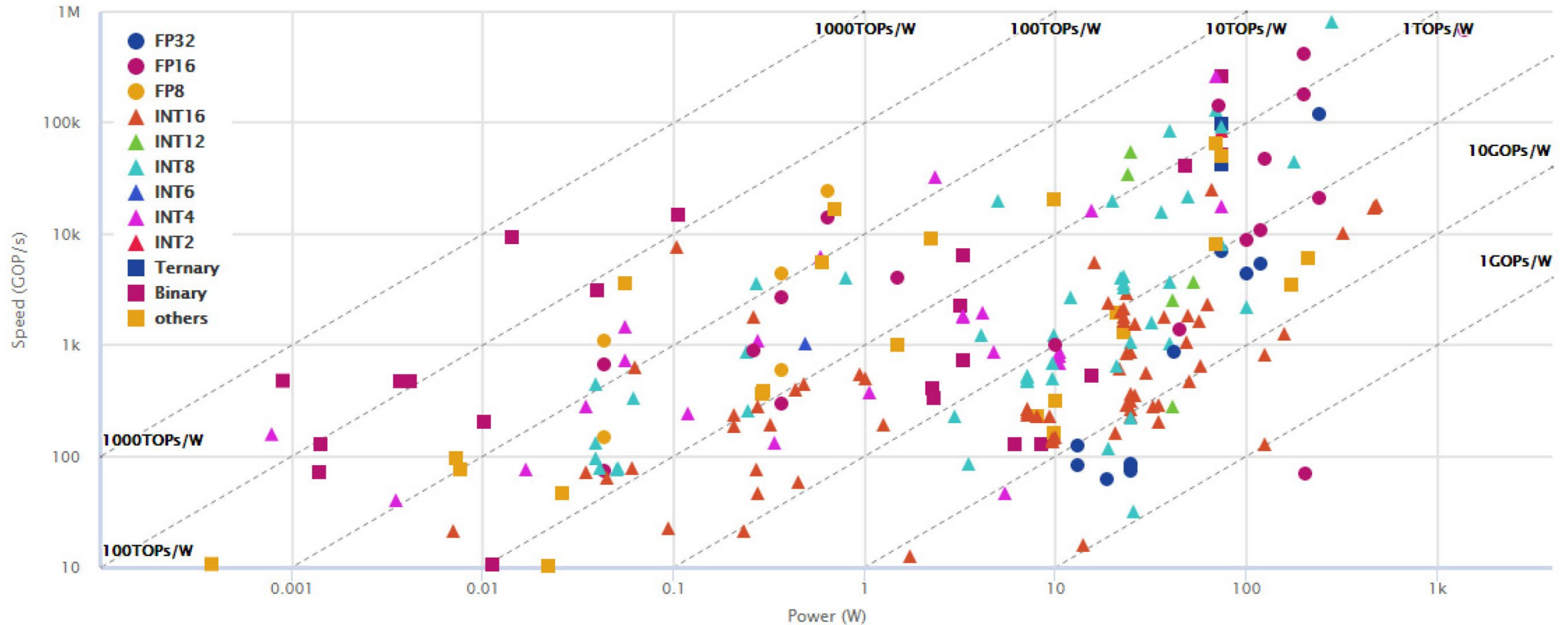
- **Quantize: Reduce size** of operands for storage/compute
 - Floating point → Fixed point
 - Bit-width reduction
 - Non-linear quantization
- **Prune: Reduce number** of operations for storage/compute
 - Network Pruning
 - Compact Network Architectures

Quantization in modern GPUs

Support for quantization in SoA GPUs (NVIDIA)

		Input Operands	Accumulator	TOPS
V100		FP32	FP32	15.7
		FP16	FP32	125
A100		FP32	FP32	19.5
		TF32	FP32	156
		FP16	FP32	312
		BF16	FP32	312
		INT8	INT32	624
		INT4	INT32	1248
		BINARY	INT32	4992
		FP64	FP64	19.5

Not only GPUs!



Highcharts.com

Source: <https://nicsefc.ee.tsinghua.edu.cn/projects/neural-network-accelerator/>

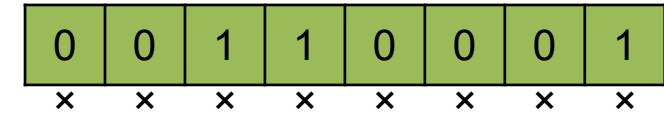
Quantization → Smaller Memory Footprint + More lightweight computation

Numeric Data Types

How is numeric data represented in modern computing systems?

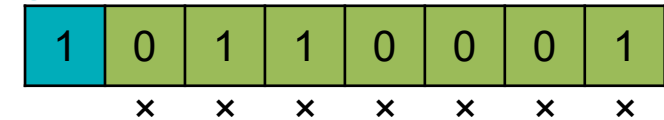
Integer

- Unsigned Integer
 - n -bit Range: $[0, 2^n - 1]$
- Signed Integer
 - Sign-Magnitude Representation
 - n -bit Range: $[-2^{n-1} - 1, 2^{n-1} - 1]$
 - Both 000...00 and 100...00 represent 0
 - Two's Complement Representation
 - n -bit Range: $[-2^{n-1}, 2^{n-1} - 1]$
 - 000...00 represents 0
 - 100...00 represents -2^{n-1}

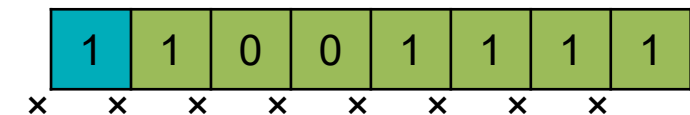


$$2^7 + 2^6 + 2^5 + 2^4 + 2^3 + 2^2 + 2^1 + 2^0 = 49$$

Sign Bit



$$- 2^6 + 2^5 + 2^4 + 2^3 + 2^2 + 2^1 + 2^0 = -49$$



$$-(2^7 + 2^6 + 2^5 + 2^4 + 2^3 + 2^2 + 2^1 + 2^0) = -49$$

Fixed-Point Number



Integer . Fraction

“Decimal” Point



x x x x x x x x

$$2^3 + 2^2 + 2^1 + 2^0 + 2^{-1} + 2^{-2} + 2^{-3} + 2^{-4} = 3.0625$$



x x x x x x x x

$$(2^7 + 2^6 + 2^5 + 2^4 + 2^3 + 2^2 + 2^1 + 2^0) \times 2^{-4} = 49 \times 0.0625 = 3.0625$$

(using 2's complement representation)

Floating-Point Number

Example: 32-bit floating-point number in IEEE 754



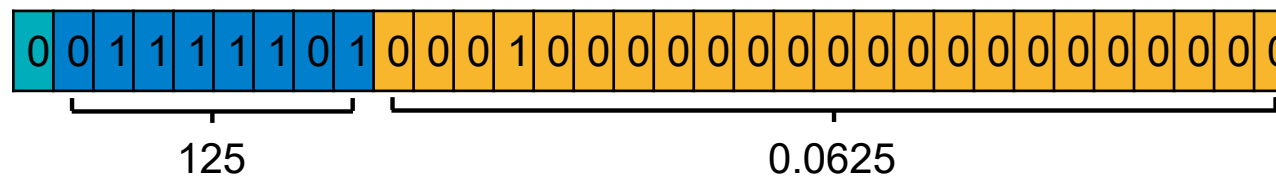
Sign 8 bit Exponent

23 bit Fraction

$$(-1)^{\text{sign}} \times (1 + \text{Fraction}) \times 2^{\text{Exponent}-127} \quad \leftarrow \quad \text{Exponent Bias} = 127 = 2^{8-1}-1$$

(significant / mantissa)

$$0.265625 = 1.0625 \times 2^{-2} = (1 + \underline{0.0625}) \times 2^{\underline{125}-127}$$



Floating-Point Number

Exponent Width → Range; Fraction Width → Precision

IEEE 754 Single Precision 32-bit Float (IEEE FP32)



IEEE Half Precision 16-bit Float (IEEE FP16)



Brain Float (BF16)



Nvidia TensorFloat (TF32)



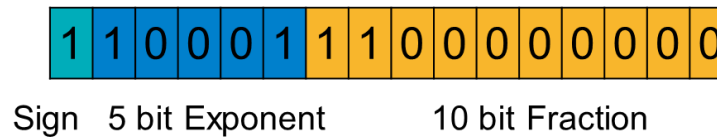
AMD 24-bit Float (AMD FP24)



Exponent (bits)	Fraction (bits)	Total (bits)
8	23	32
5	10	16
8	7	16
8	10	19
7	16	24

Numeric Data Types

- **Question:** What is the following IEEE half precision (IEEE FP16) number in decimal?



Exponent Bias = 15_{10}
15 in base 10

- Sign: -
- Exponent: $10001_2 - 15_{10} = 17_{10} - 15_{10} = 2_{10}$
- Fraction: $1100000000_2 = 0.75_{10}$
- Decimal Answer = $-(1 + 0.75) \times 2^2 = -1.75 \times 2^2 = -7.0_{10}$

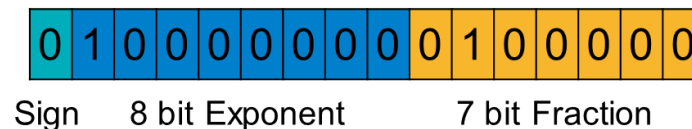
Numeric Data Types

- **Question:** What is the decimal 2.5 in Brain Float (BF16)?

$$2.5_{10} = 1.\underline{25}_{10} \times 2^1$$

Exponent Bias = 127_{10}

- Sign: +
- Exponent Binary: $1_{10} + 127_{10} = 128_{10} = 10000000_2$
- Fraction Binary: $0.25_{10} = 0100000_2$
- Binary Answer

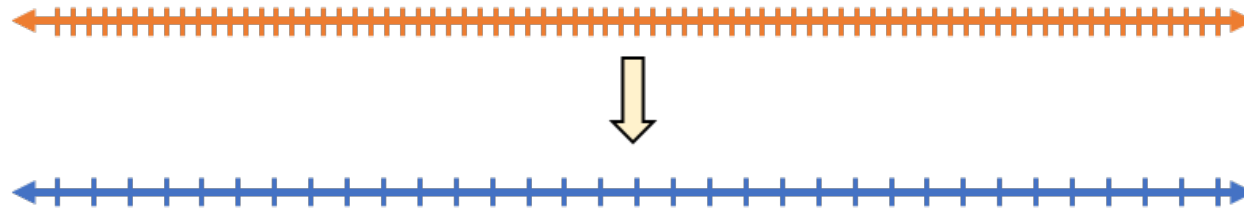


Quantization



Quantization

Reduce the precision for storing model weights and activations



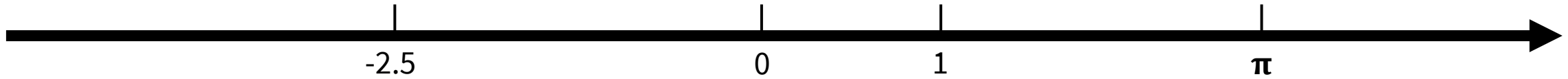
Using smaller float formats (e.g., 16-bit mini-float)

Using low-precision integers (e.g., int8)

Using other non-uniform data formats (e.g., log-domain quantization).

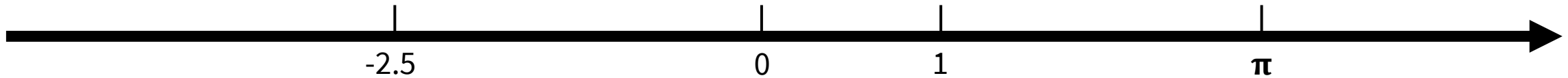
Representing *real* numbers: floating-point vs fixed-point

Real numbers: $r \in \mathbb{R}$

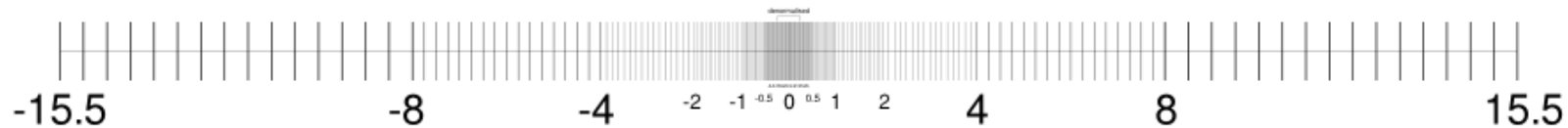


Representing *real* numbers: floating-point vs fixed-point

Real numbers: $r \in \mathbb{R}$



Floating-point representation: $r \sim s \times b^e \in \mathbb{R}, s, b, e \in \mathbb{Z} \rightarrow s, b, e$ are represented explicitly (you need bits)



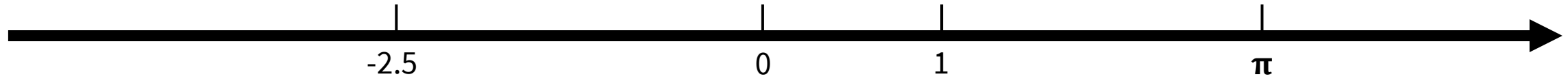
e controls **range**

s controls **precision**

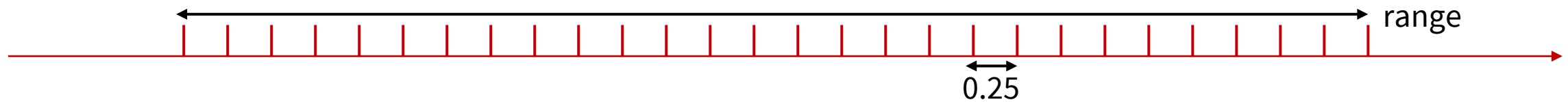


Representing *real* numbers: floating-point vs fixed-point

Real numbers: $r \in \mathbb{R}$



Fixed-point representation: $r \sim i \times 2^q \in \mathbb{R}, i, q \in \mathbb{Z} \rightarrow q$ is implicitly known by the application
 $\rightarrow i$ is an INT!



For *fixed-point*, you need only **integer** data and operations

$$r_1 \times r_2 = i_1 \times 2^{q_1} \times i_2 \times 2^{q_2} = i_1 \times i_2 \times 2^{q_1+q_2} \quad \text{exponents are implicit}$$

INT produces constant representation error and would be nice from an engineering point of view, but why do we prefer FLOATING points, that has variable error because you can get a larger range (int: 2^{32} vs float:).
This is why we use them in neural networks, it helps avoiding numerical issues.



Can we use a “fixed-point” representation for DNNs? If so, how many bits?

- DNN tensors tend to have a **predictable range** at inference time
 - Weights are fixed after training → range known at design time
 - Activation tensors depend on input → range can be evaluated statistically (**calibration**)
 - At training time, gradients *do not* typically have predictable range (varies along the training procedure)



Can we use a “fixed-point” representation for DNNs? If so, how many bits?

- DNN tensors tend to have a **predictable range** at inference time
 - Weights are fixed after training → range known at design time
 - Activation tensors depend on input → range can be evaluated statistically (**calibration**)
 - At training time, gradients *do not* typically have predictable range (varies along the training procedure)
- How much **precision** is sufficient to represent weights and activations?
 - DNNs are (supposed to be) fairly tolerant to *noise*
 - **16** bits → **65536** representable levels → smallest value in [0,1] range is **0.0000153**



Can we use a “fixed-point” representation for DNNs? If so, how many bits?

- DNN tensors tend to have a **predictable range** at inference time
 - Weights are fixed after training → range known at design time
 - Activation tensors depend on input → range can be evaluated statistically (**calibration**)
 - At training time, gradients *do not* typically have predictable range (varies along the training procedure)
- How much **precision** is sufficient to represent weights and activations?
 - DNNs are (supposed to be) fairly tolerant to *noise*
 - **16** bits → **65536** representable levels → smallest value in $[0,1]$ range is **0.0000153**
 - **8** bits → **256** representable levels → smallest value in $[0,1]$ range is **0.0039** → **enough?**



Can we use a “fixed-point” representation for DNNs? If so, how many bits?

- DNN tensors tend to have a **predictable range** at inference time
 - Weights are fixed after training → range known at design time
 - Activation tensors depend on input → range can be evaluated statistically (**calibration**)
 - At training time, gradients *do not* typically have predictable range (varies along the training procedure)
- How much **precision** is sufficient to represent weights and activations?
 - DNNs are (supposed to be) fairly tolerant to *noise*
 - **16** bits → **65536** representable levels → smallest value in $[0,1]$ range is **0.0000153**
 - **8** bits → **256** representable levels → smallest value in $[0,1]$ range is **0.0039** → **enough?**
 - **4** bits → **16** representable levels → smallest value in $[0,1]$ range is **0.067** → **not enough, right?**
By increasing the number of bits we are getting one order of magnitude more each time of possible error



Can we use a “fixed-point” representation for DNNs? If so, how many bits?

- DNN tensors tend to have a **predictable range** at inference time
 - Weights are fixed after training → range known at design time
 - Activation tensors depend on input → range can be evaluated statistically (**calibration**)
 - At training time, gradients *do not* typically have predictable range (varies along the training procedure)
- How much **precision** is sufficient to represent weights and activations?
 - DNNs are (supposed to be) fairly tolerant to *noise*
 - **16** bits → **65536** representable levels → smallest value in $[0,1]$ range is **0.0000153**
 - **8** bits → **256** representable levels → smallest value in $[0,1]$ range is **0.0039** → **enough?**
 - **4** bits → **16** representable levels → smallest value in $[0,1]$ range is **0.067** → **not enough, right?**

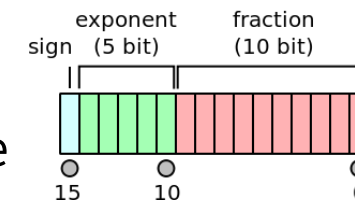
But: DNNs can be **retrained and **tuned** to execution with low bitwidth**



Quantization Formats

16-bit minifloats (aka half-precision floats) basically come for free:

- Can be used for both training and inference with no drop in accuracy in most cases
- But they're still floats
- Increasingly adopted for **training/inference** in the cloud. Less common at the edge



Quantization for inference

Int-8 is the current standard at the edge:

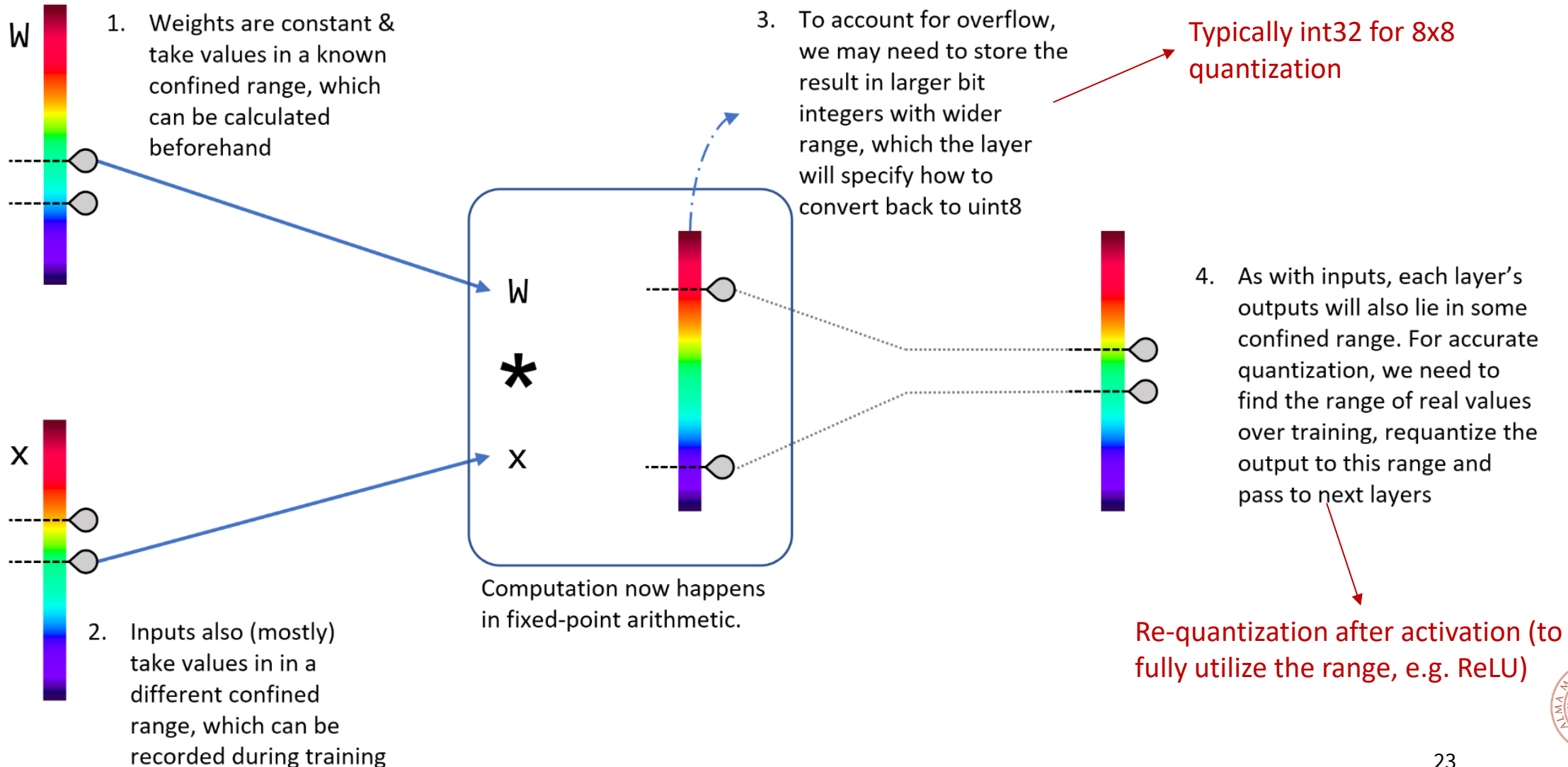
- Drop in accuracy often minimal (especially with Quantization-Aware Training).
- Most ISAs have byte-level instructions (although not always byte-level SIMD arithmetic)

Sub-byte integer quantization is attracting attention:

- 1-bit quantization (BNN) is already popular for simple tasks
- 4-bit is increasingly used, despite its packing/unpacking overheads.

Quantization

Example: int8 integer quantization for inference



Quantization Benefits

Storage and memory:

- Reduce ROM/disk occupation
- Reduce RAM occupation
- Increase data-transfers bandwidth (e.g., with SIMD load/store)

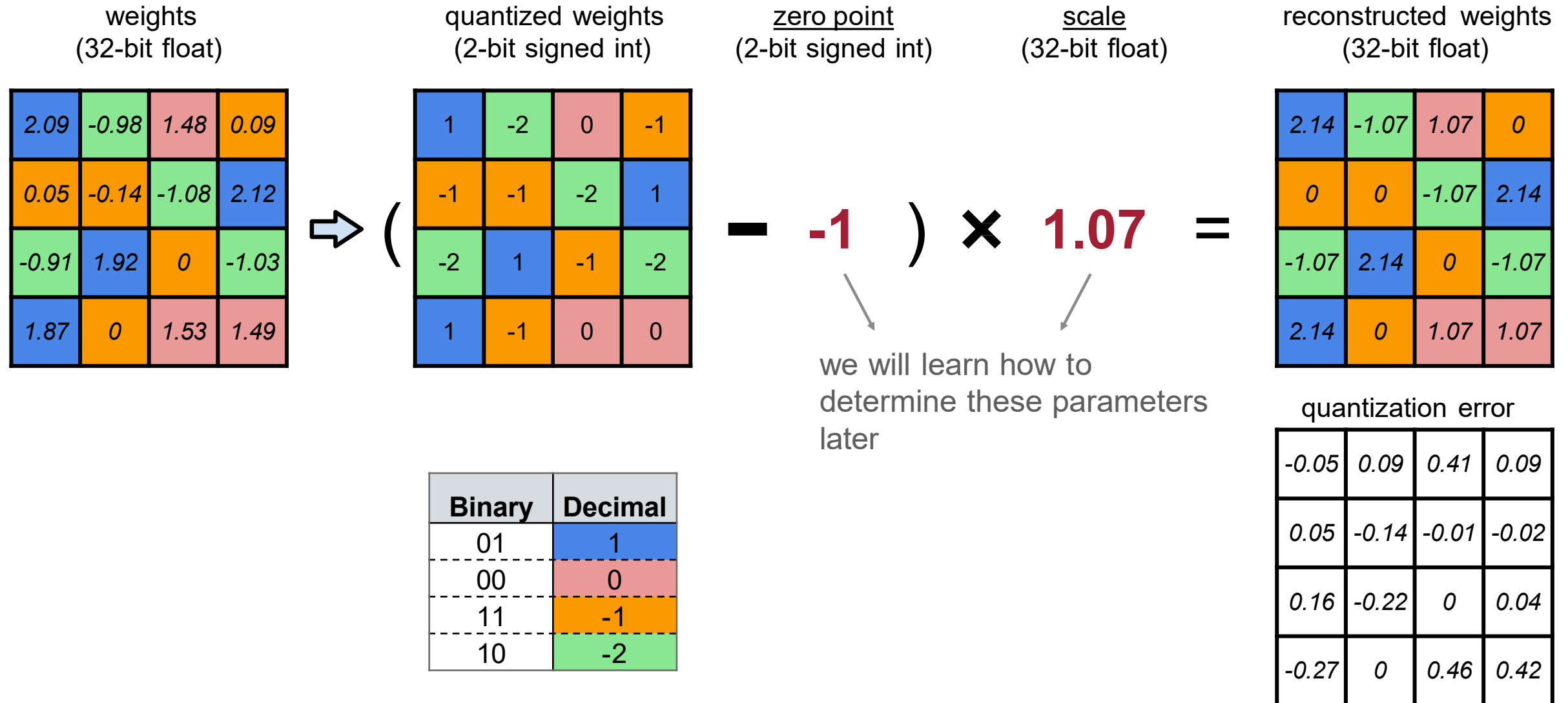
Compute:

- Speed-up thanks to ALU vs FPU
- Further speed-up thanks to SIMD operations
- Enable deployment on FPU-less Microcontrollers



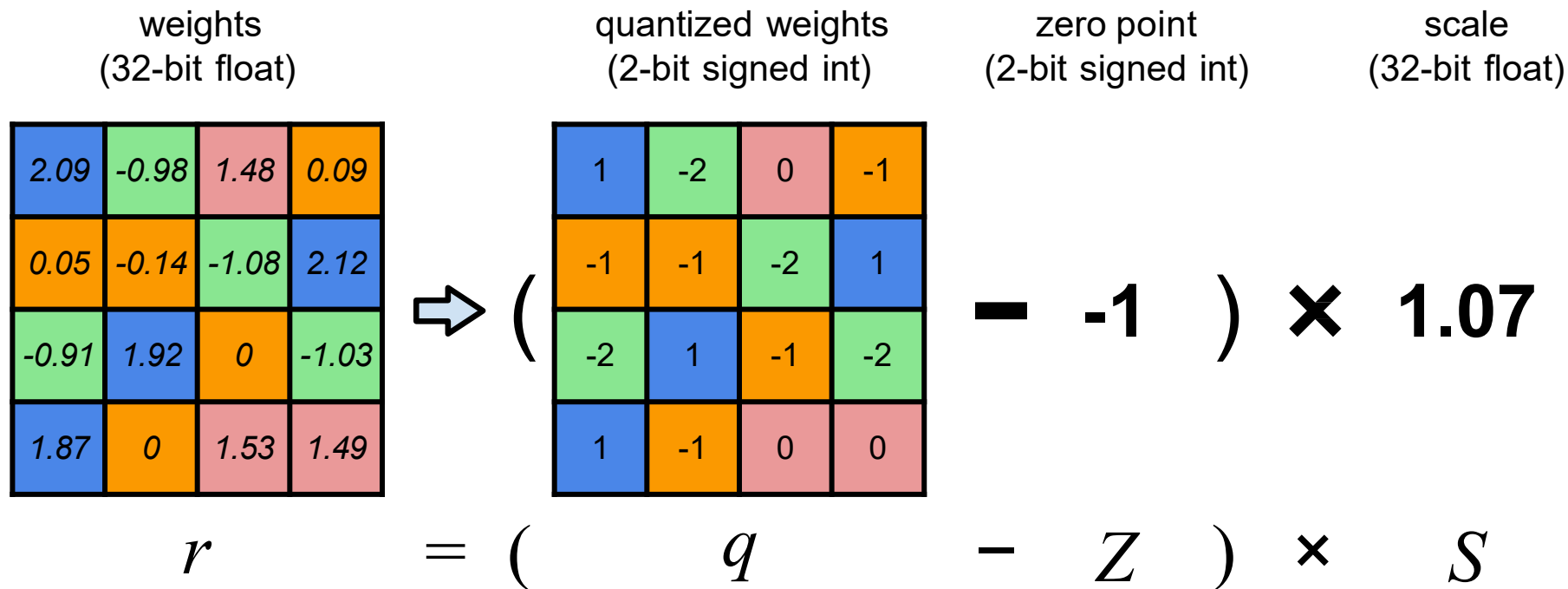
Linear Quantization: what is it?

An affine mapping of integers to real numbers



Linear Quantization

An affine mapping of integers to real numbers $r = S(q - Z)$



Floating-point

Integer

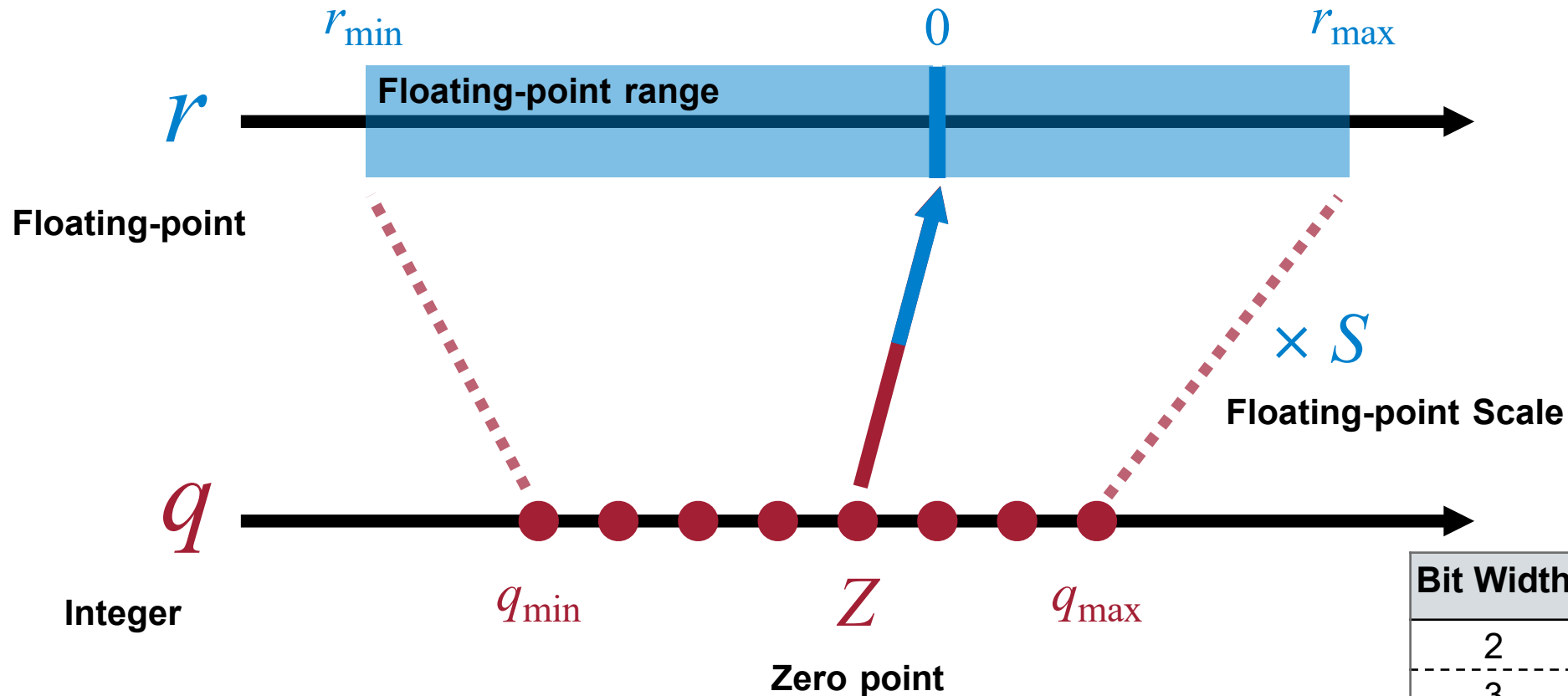
Integer

Floating-point

- quantization parameter
- allow real number $r=0$ be exactly representable by a quantized integer
- quantization parameter

Linear Quantization

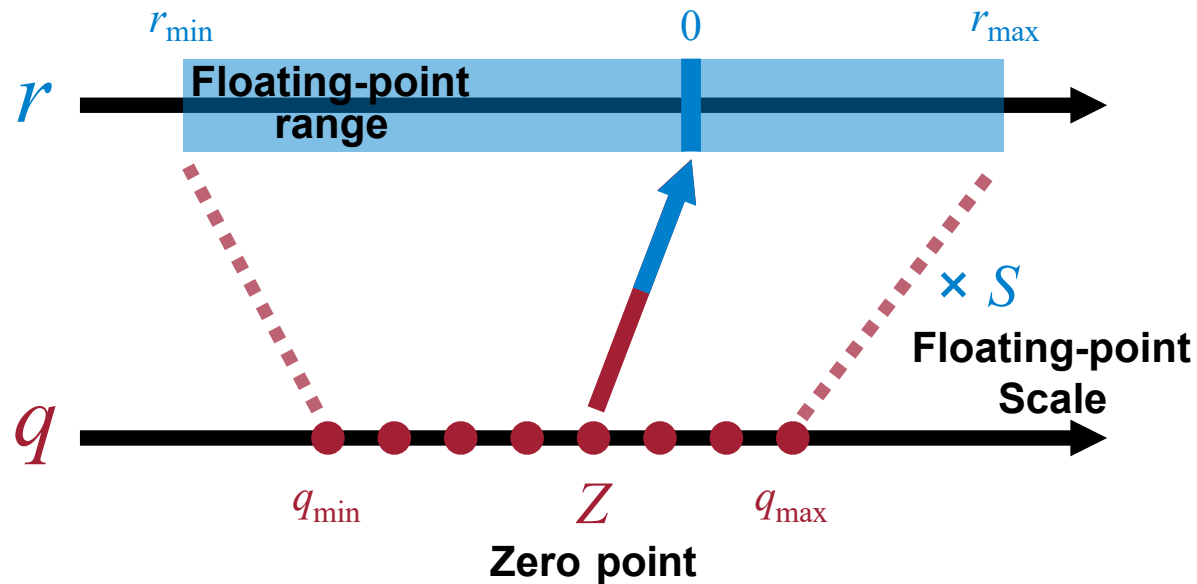
An affine mapping of integers to real numbers $r = S(q - Z)$



Bit Width	$q_{\min}$	$q_{\max}$
2	-2	1
3	-4	3
4	-8	7
N	-2^{N-1}	$2^{N-1}-1$

Scale of Linear Quantization

Linear Quantization is an affine mapping of integers to real numbers $r = S(q - Z)$



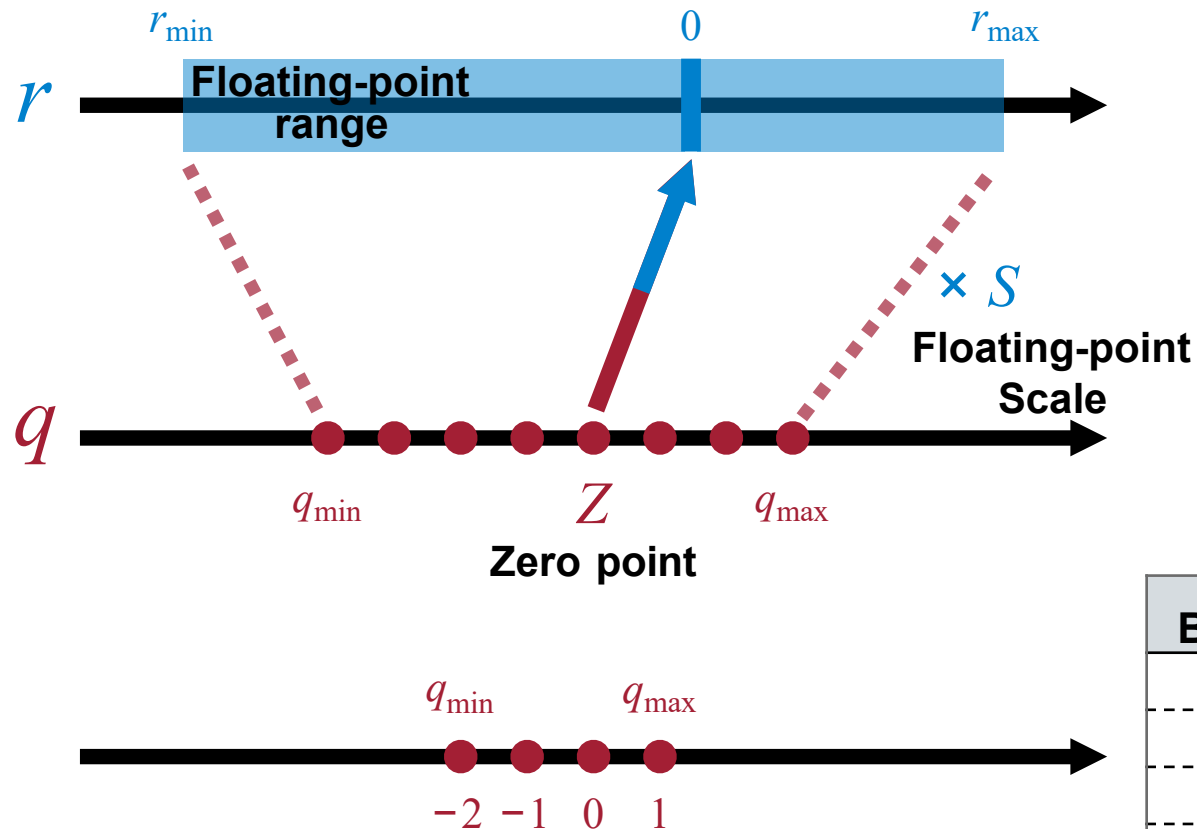
$$\begin{aligned} r_{\max} &= S(q_{\max} - Z) \\ r_{\min} &= S(q_{\min} - Z) \end{aligned} \quad \ominus$$

$$\downarrow$$
$$r_{\max} - r_{\min} = S(q_{\max} - q_{\min})$$

$$S = \frac{r_{\max} - r_{\min}}{q_{\max} - q_{\min}}$$

Scale of Linear Quantization

Linear Quantization is an affine mapping of integers to real numbers $r = S(q - Z)$



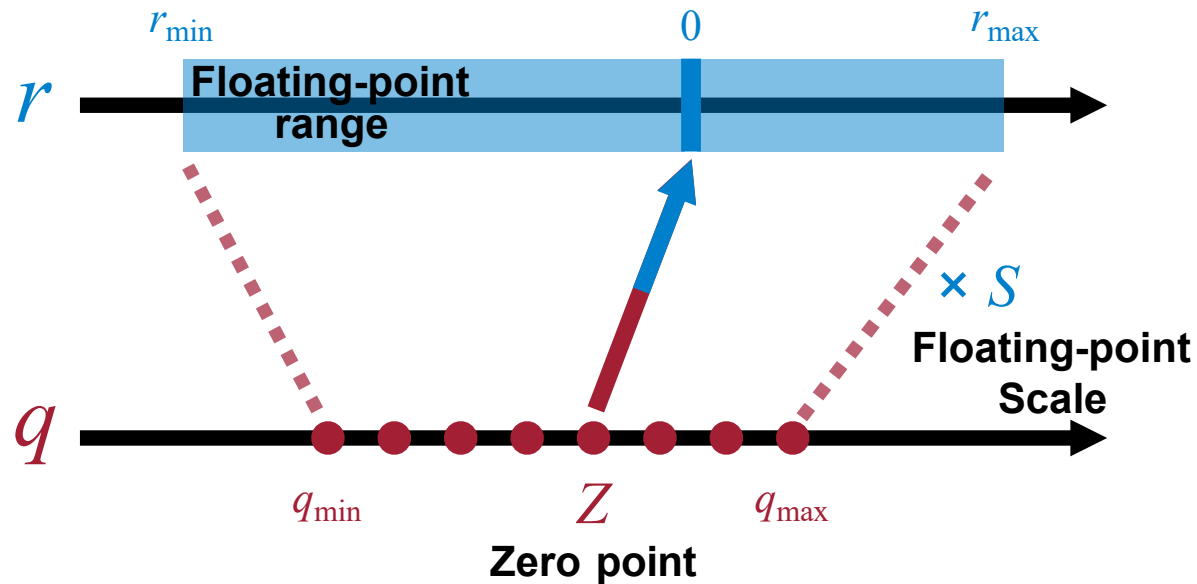
2.09	-0.98	1.48	0.09
0.05	-0.14	-1.08	2.12
-0.91	1.92	0	-1.03
1.87	0	1.53	1.49

$$S = \frac{r_{\max} - r_{\min}}{q_{\max} - q_{\min}}$$
$$= \frac{2.12 - (-1.08)}{1 - (-2)}$$
$$= 1.07$$

Binary	Decimal
01	1
00	0
11	-1
10	-2

Zero Point of Linear Quantization

Linear Quantization is an affine mapping of integers to real numbers $r = S(q - Z)$



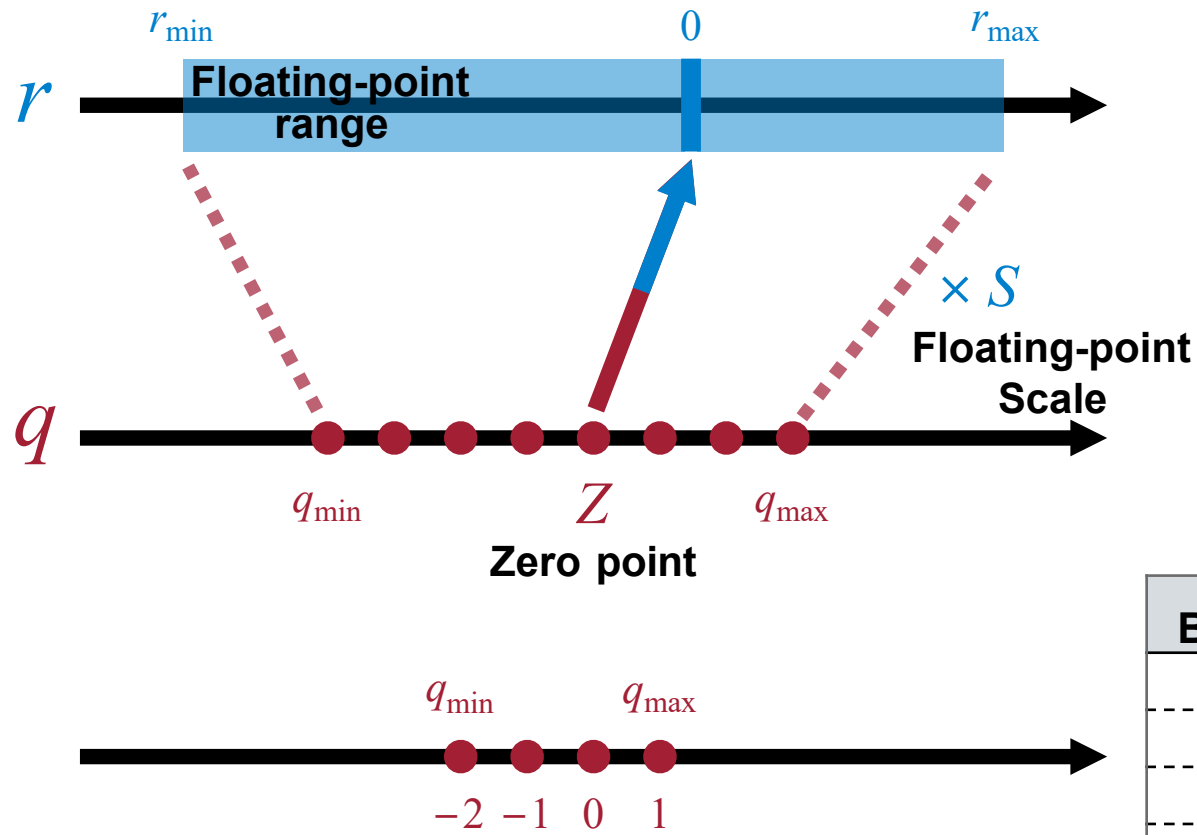
$$r_{\min} = S(q_{\min} - Z)$$

$$Z = q_{\min} - \frac{r_{\min}}{S}$$

$$Z = \text{round} \left(q_{\min} - \frac{r_{\min}}{S} \right)$$

Zero Point of Linear Quantization

Linear Quantization is an affine mapping of integers to real numbers $r = S(q - Z)$



2.09	-0.98	1.48	0.09
0.05	-0.14	-1.08	2.12
-0.91	1.92	0	-1.03
1.87	0	1.53	1.49

$$Z = q_{\min} - \frac{r_{\min}}{S}$$

Binary	Decimal
01	1
00	0
11	-1
10	-2

$$\begin{aligned} &= \text{round}\left(-2 - \frac{-1.08}{1.07}\right) \\ &= -1 \end{aligned}$$

Int Linear (Affine) Quantizer: How to use it

Commonly use format (e.g., TFLite)

Asymmetric integer quantization, to deal with **non-centered** distributions of activations (e.g. ReLU).

$$int8_value = round(\frac{real_value + z}{s})$$

Exact representation of 0.0 (z) is beneficial for accuracy:

- Zero is a “special” value for DNNs (padding, dead neuron, etc).

Normally, **z is fixed to 0 for weights.**



Integer Quantization Formats

Compute S and z for a given tensor (**a**)

- Basic: map the entire range ($a_{\min}, a_{\max}$)

Activations	$S = \frac{a_{\max} - a_{\min}}{2^{Nbit} - 1}$	$z = \frac{a_{\max} - a_{\min}}{2}$
Weights	$S = \frac{2 * \max(w_{\max} , w_{\min})}{2^{Nbit} - 1}$	$z = 0$

Some tools, like TFLite prefer to use symmetric two's complement values, e.g. [-127, 127] for 8bit

- To compute S and z you need to know the smallest and largest value you deal with
- In general for activations, the **range can only be estimated** (using a sample of data). Then, re-quantization of output activations needs an additional saturation step:

$$int8_{value} = clamp\left(\frac{real_{value} + z}{S}, -127, 127\right)$$

- Some tools use a smaller S for weights too, saturating large values in exchange for more precision around z. Sometimes (empirically) more accurate.



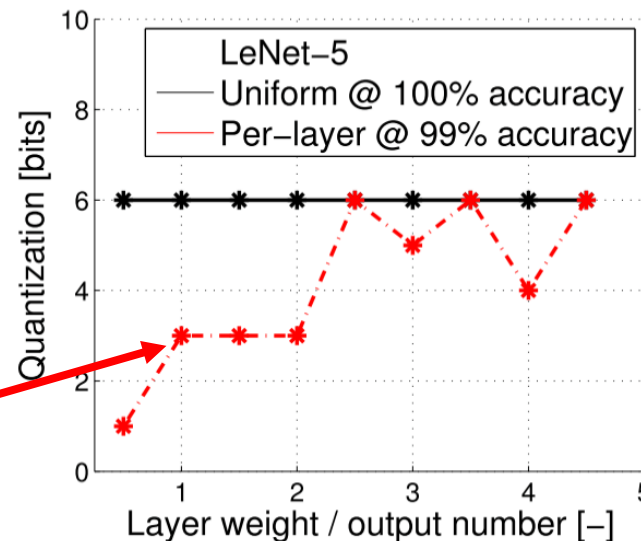
Integer Quantization: Fixed- vs Mixed-Precision

Fixed-precision: while S and z change per-tensor (or channel), the bit-width N is fixed for the entire network

Mixed-precision: uses a different N layer-wise or channel-wise.

- N_w (weights) can also be different from N_x (activations)
- Possibly higher compression at iso-accuracy
- Various search algorithms (greedy, simulated annealing, genetic) to determine the optimal N

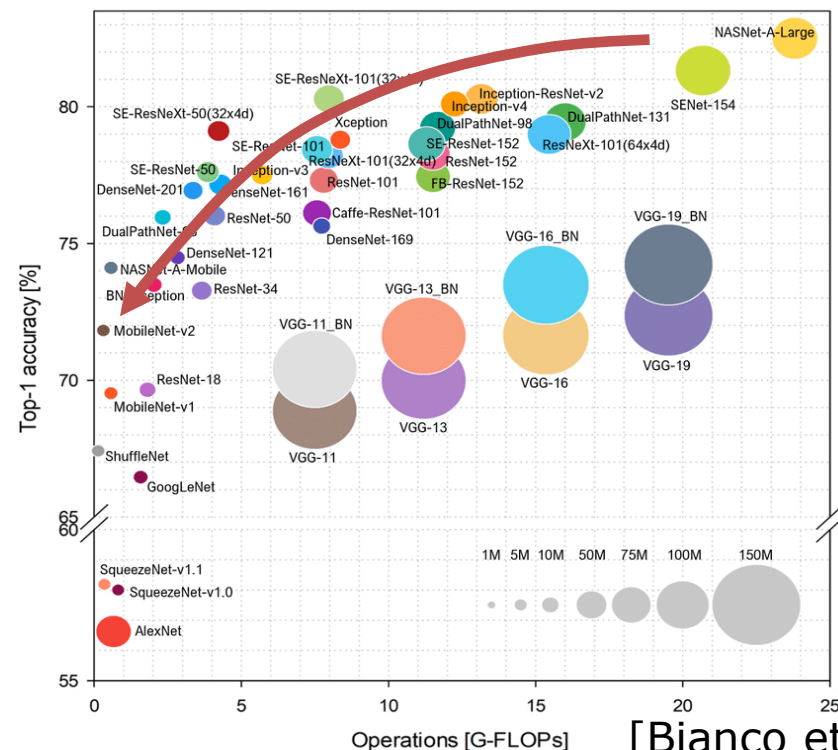
Using per-layer quantization I only loose 1% in accuracy, but I reduce significantly the footprint of the network



[Source] B. Moons. Energy-efficient ConvNets through approximate computing, 2016



Reduced Precision DL Models: An Example



[Bianco et al. IEEE Access, 2018]

Quantized MobileNet-v1 model

Quantization Method	Top1 Accuracy	Weight Memory Footprint
Full-Precision	70.9%	16.27 MB
INT-8	70.1% 0.8%	4.06 MB 4x
INT-4	66.46% 4.4%	2.35 MB 7x
Mixed-Precision	68% 2.9%	2.09 MB 8x

[Rusci et al., PML&S (2020)]

- MVM workload dominates (massive parallelism)
- Strong resiliency to Quantization (low-precision integer arithmetic is ok)
- Mixed-precision quantized neural networks (QNNs) are the natural target for Edge AI



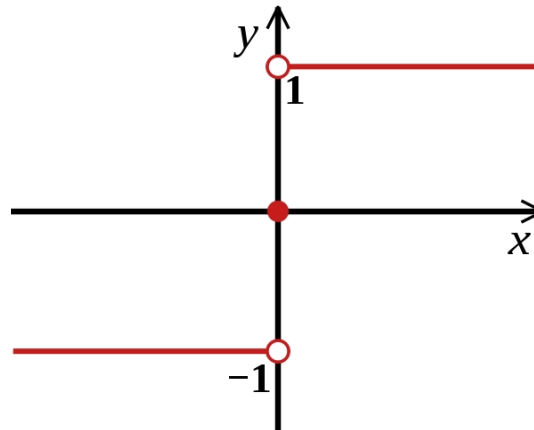
Binary Neural Networks

Extreme form of quantization, where data are constrained to have a binary value.

- Initially used only for **weights**, ([BinaryConnect](#)) then extended also to **activations** ([XNOR-Net](#))

In the simplest form, weights and activations are constrained **to** $\{-1, 1\}$

- In practice, **-1 is represented by logic-0** for HW simplicity.
- More advanced variants use $\{-\alpha, \alpha\}$, where α is set per-tensor.
- Quantization obtained with a **sign()** function



Binary Neural Networks: Pros and Cons

32x reduction in model size

MAC operations can be **replaced by bit manipulations** (XNOR + Popcount).

- Significant speed-up, also on general purpose devices

Mult is a XNOR

A	B	A * B	Logic A	Logic B	XNOR
-1	-1	1	0	0	1
-1	1	-1	0	1	0
1	-1	-1	1	0	0
1	1	1	1	1	1

Accumulate is a Popcount

p = count of 1s

n = count of 0s (-1s),

$$Accum = p - n = p - (N - p) = 2 * p - N$$

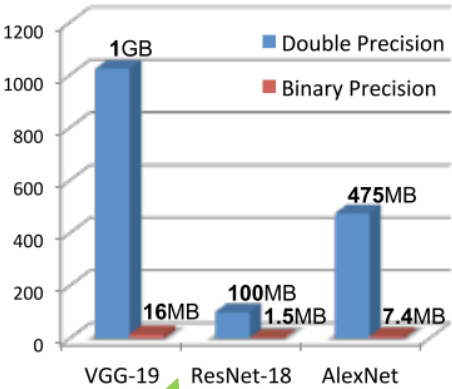
Since 2 and N are constants, equivalent to p

Significant accuracy drop on complex tasks.



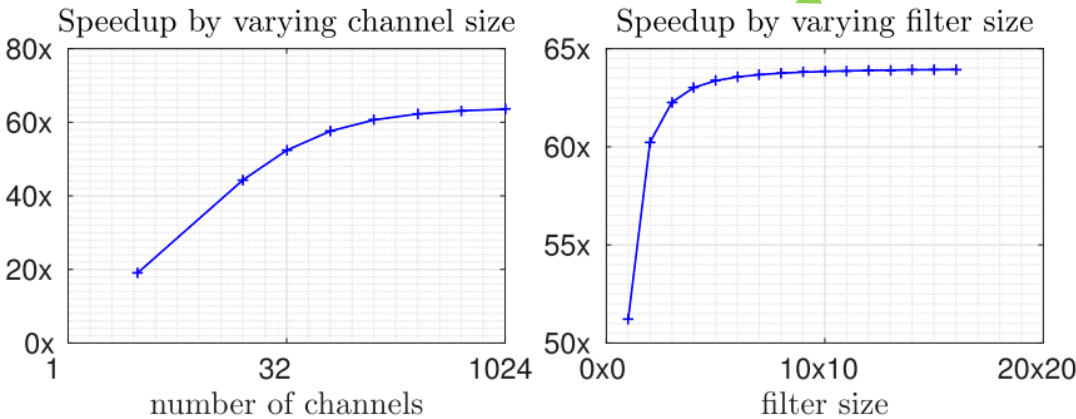
Binary Neural Networks: Pros and Cons

Memory Occupation



Significant Footprint Reduction

Speed-up

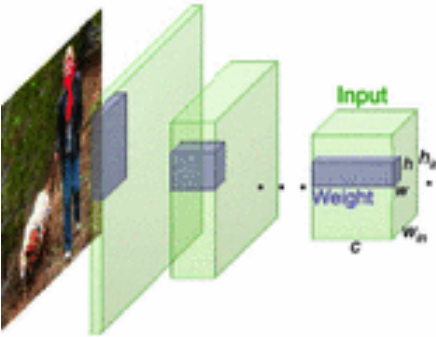


Significant Computation Speed-up

Accuracy

	Network Variations		Operations used in Convolution	Memory Saving (Inference)	Computation Saving (Inference)	Accuracy on ImageNet (AlexNet)
Standard Convolution	Real-Value Inputs 0.11 -0.21 -0.34 -0.25 0.61 0.52	Real-Value Weights 0.02 -0.2 -0.41 3.2 0.5 -0.44	$+, -, \times$	1x	1x	%56.7
Binary Weight	Real-Value Inputs 0.11 -0.21 -0.34 -0.25 0.61 0.52	Binary Weights 1 1 1 1 1 1	$+, -$	~32x	~2x	%56.8
BinaryWeight Binary Input (XNOR-Net)	Binary Inputs 1 -1 -1 -1 1 1	Binary Weights 1 1 1 1 1 1	XNOR, bitcount	~32x	~58x	%44.2

..but significant accuracy drop as well



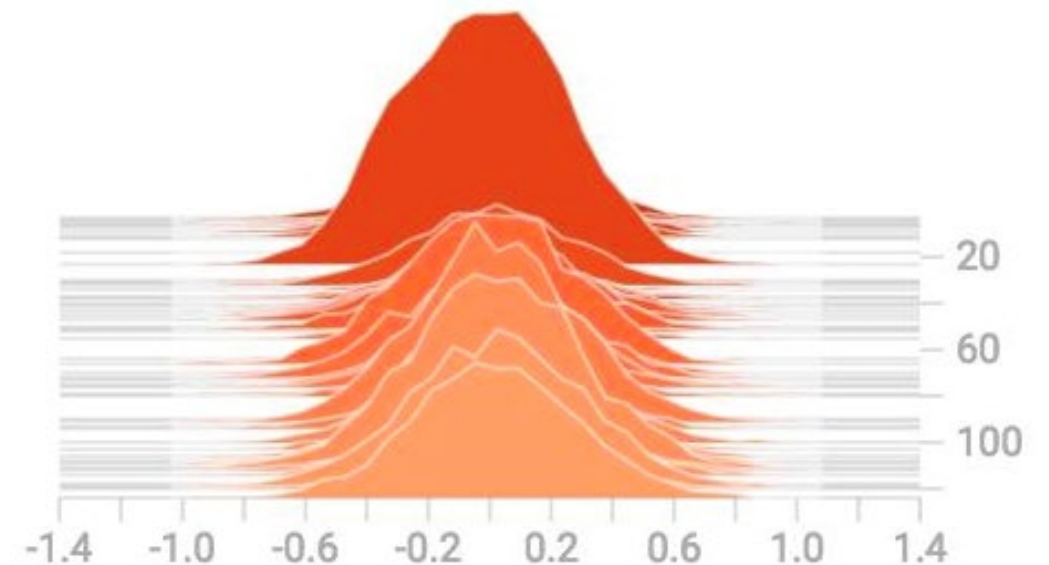
Non-Uniform Quantization

Distributions of weights and activations are almost never uniform.

Non-uniformly spaced quantization points may improve accuracy

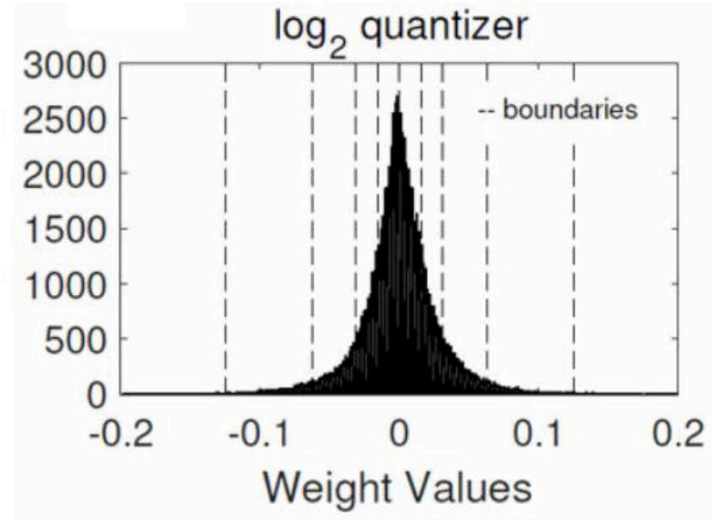
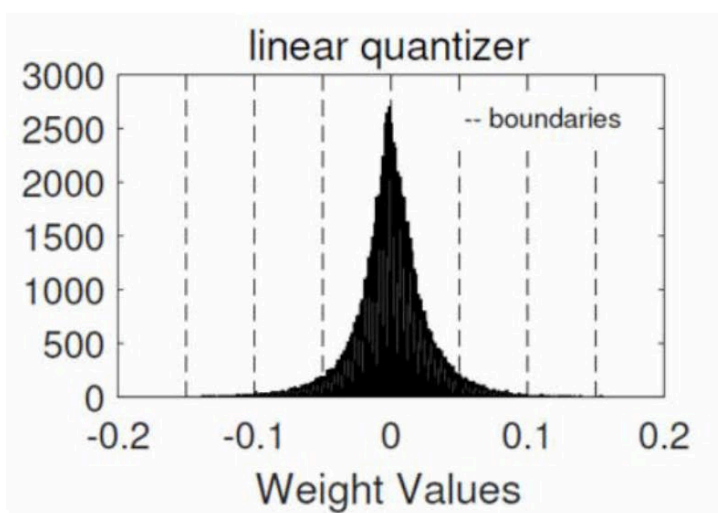
Two representative examples:

- Log-domain quantization
- Learned quantization (*weight sharing* or clustering).



Log-domain quantization

More quantization levels close to 0, since that is where most weight and activations values fall.



[Source] E. H. Lee et al. LogNet

With log2 encoding, multiplications can be implemented by shifts.

- $\tilde{w}_i = \text{Quant}(\log_2(w_i)), \tilde{x}_i = \text{Quant}(\log_2(x_i))$
- $w^T x \cong \sum_{i=1}^n 2^{\tilde{w}_i + \tilde{x}_i} = \sum_{i=1}^n \text{Bitshift}(1, \tilde{w}_i + \tilde{x}_i)$

Returning to logarithmic space simply involves finding the leftmost 1 in the result (+ rounding)



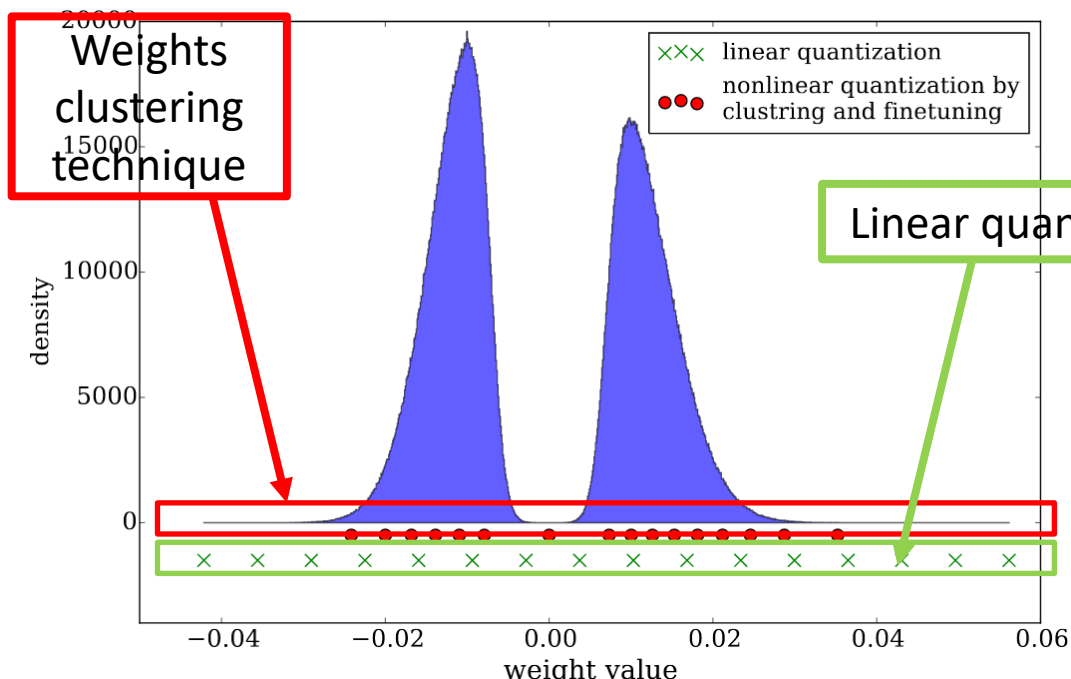
Weights Clustering

A form of quantization in which rather than reducing the precision of each weight/activation value, we reduce the **number of distinct values allowed**.

- Store the allowed values in a **codebook** (or *look-up table*)
- Store weight as codebook index $\rightarrow$ reduce storage size

Compute **weights clusters** and store centroids + cluster indexes.

- Original storage: $4 \times 4 \times 32\text{bit} = 512\text{bit}$
- Clustered storage: $4 \times 32\text{bit}$ (centroids) + $4 \times 4 \times \log_2(4)$ (indexes) = 160bit
- **Centroids can be float or quantized**

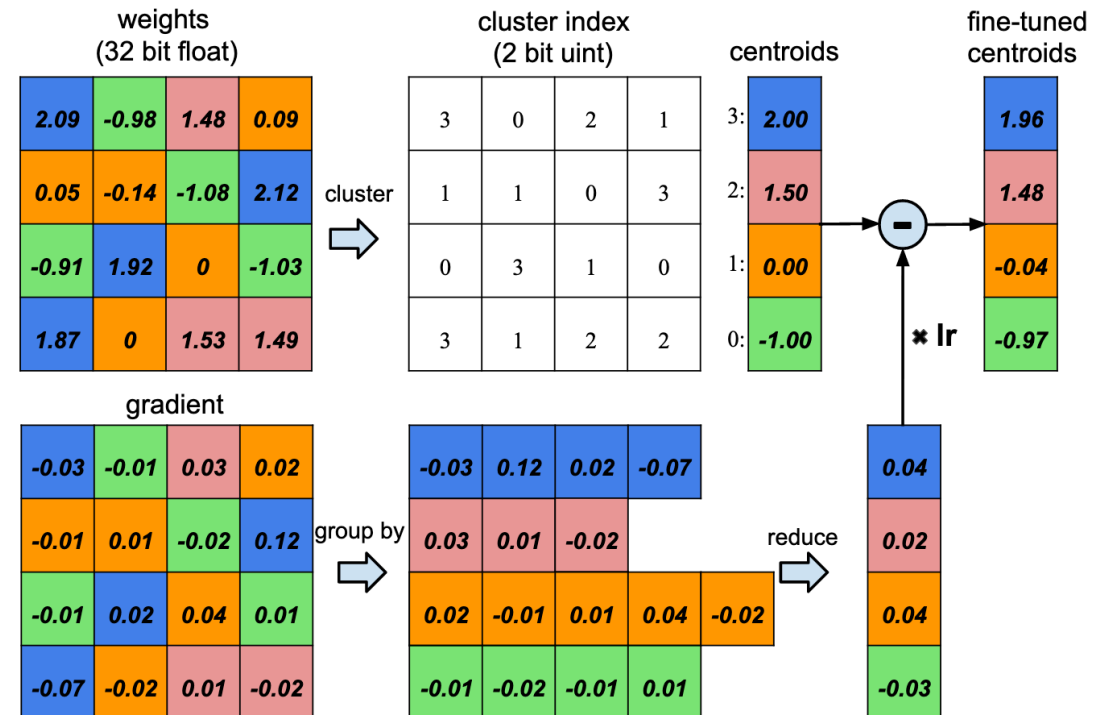
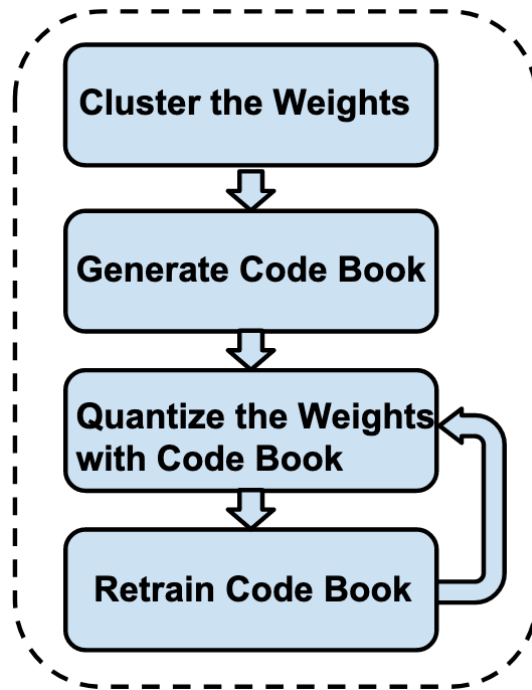


[Source] S. Han et al. Deep Compression

Weights Clustering

Centroids are set during training. The original algorithm by S. Han et al:

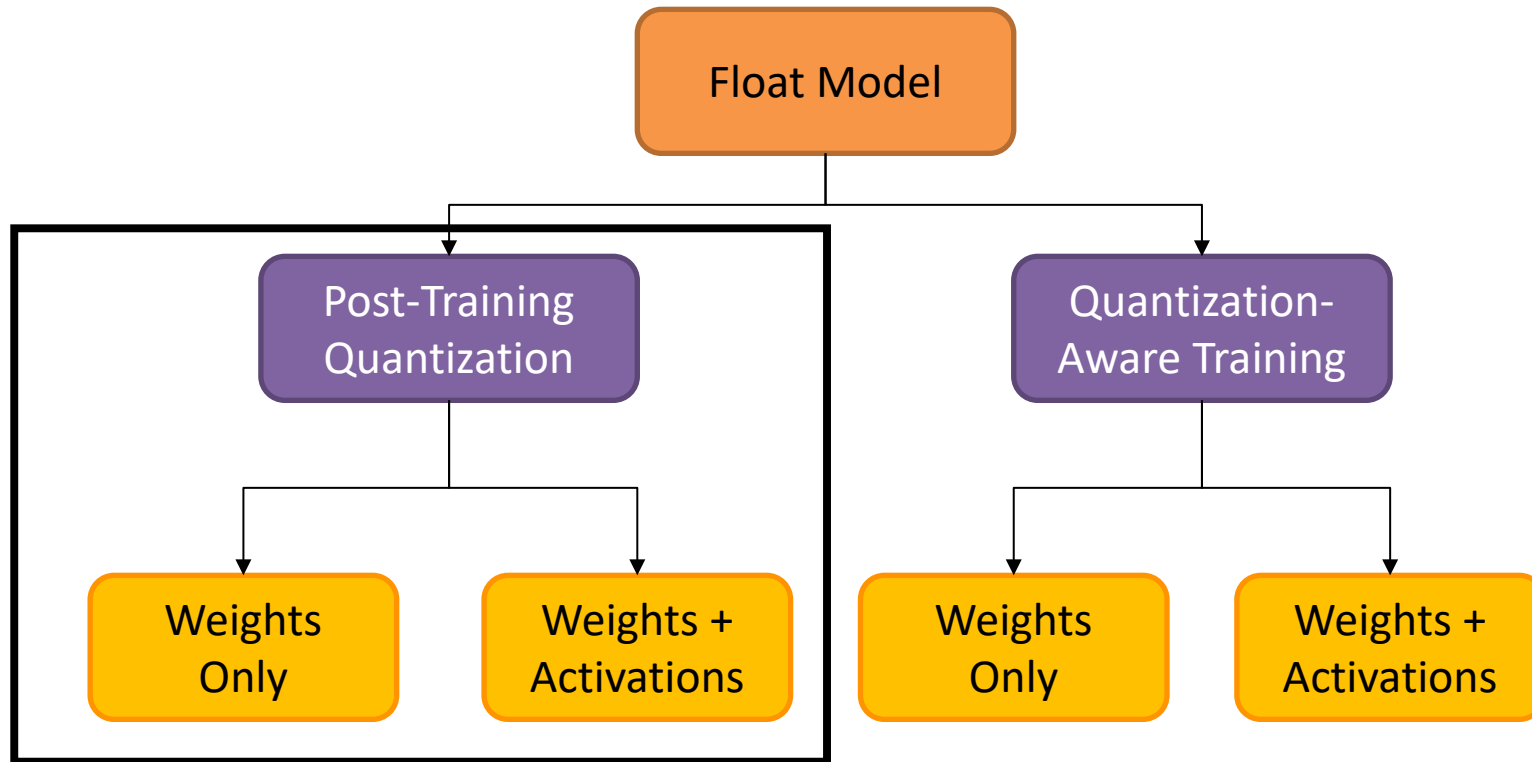
- Initializes centroids with a k-means on the weights of a pre-trained model.
- Then fine-tunes the centroids, using the total gradient of all weights belonging to the same cluster during back-prop.



[Source] S. Han et al. Deep Compression

Quantization: Strategies

Quantization can be applied either post-training or at training time.

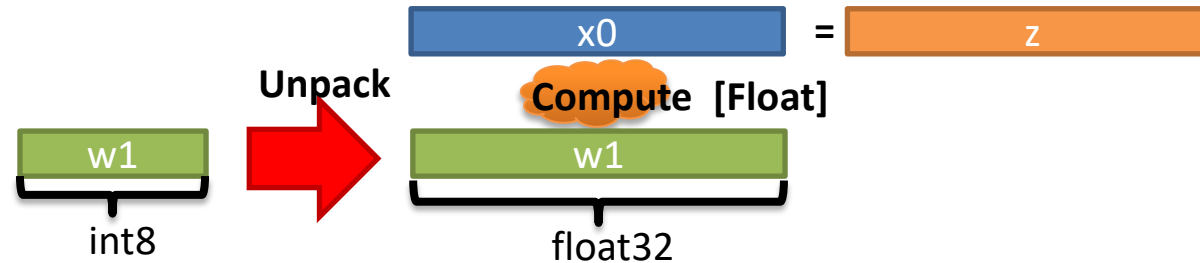


Preferred as you don't have to retrain the network

Weights-Only Quantization

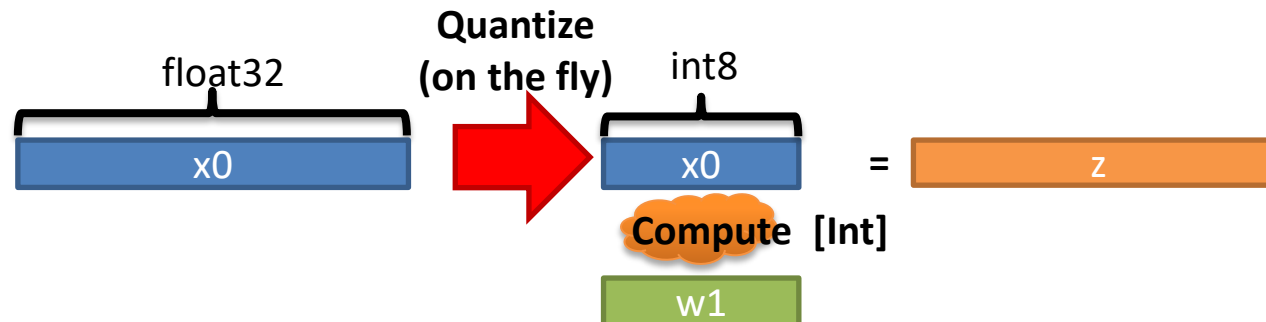
Basic approach: unpack weights to float32 on the fly for computation

- Some implementations (e.g., TFLite) cache the entire unpacked weights in RAM
- Quantization **benefits limited to storage** and (maybe) memory transfers



Dynamic range quantization: quantize activations on the fly (computing S and z)

- Activations still stored as float32. **Operations on quantized data.**



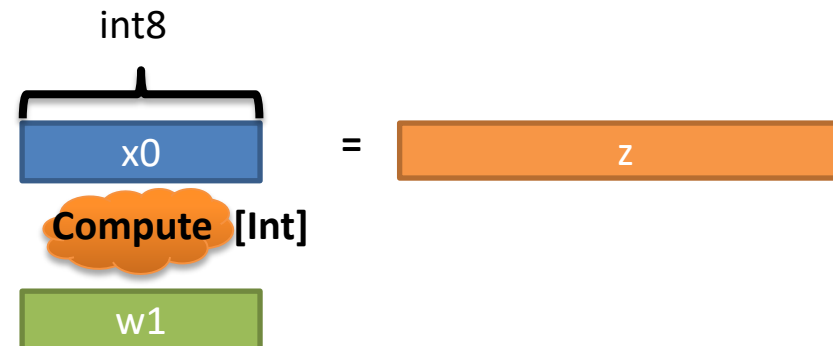
Weights+Activations Quantization

Use quantized format everywhere

- **Storage and computation, weights and activations.**

To avoid quantizing “on the fly”, S and z should be pre-computed also for activations

- Requires a subset of representative inputs,
- Typically, a small sample (100—1000) inputs is sufficient.
- Outliers can be squashed within the range with clamp()



Post-Training Quantization vs Quantization-Aware Training

Post-training Quantization:

- Simplest approach.
- **No training resources needed**
- No input data needed (for weights-only)
- May cause an **accuracy drop** on complex tasks (1-10%)

Quantization-Aware Training (QAT):

- **Requires retraining** → might be expensive
- “Shows” the approximation of weights and activations to the training algorithm
- Can yield **higher accuracy**, especially on small models.



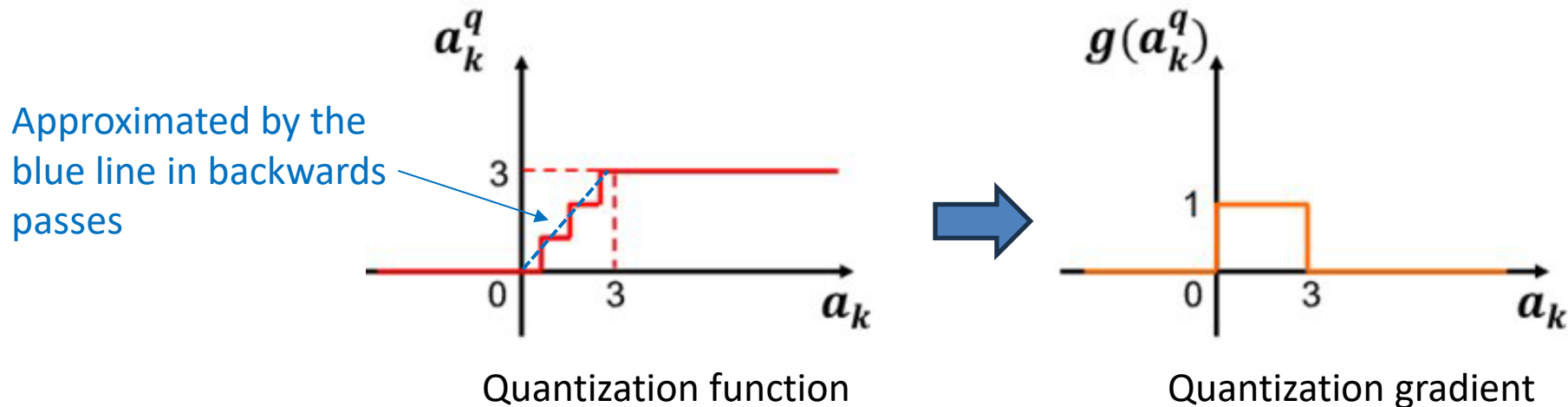
Quantization-Aware Training

Quantization complicates back-propagation of gradients during training:

- Issue: The gradient of the “re-quantization” function is zero in all differentiable points.
- Practically, quantization makes small weight updates impossible, and therefore training to convergence more difficult.

Solution: **Straight-Through Estimators** (STEs):

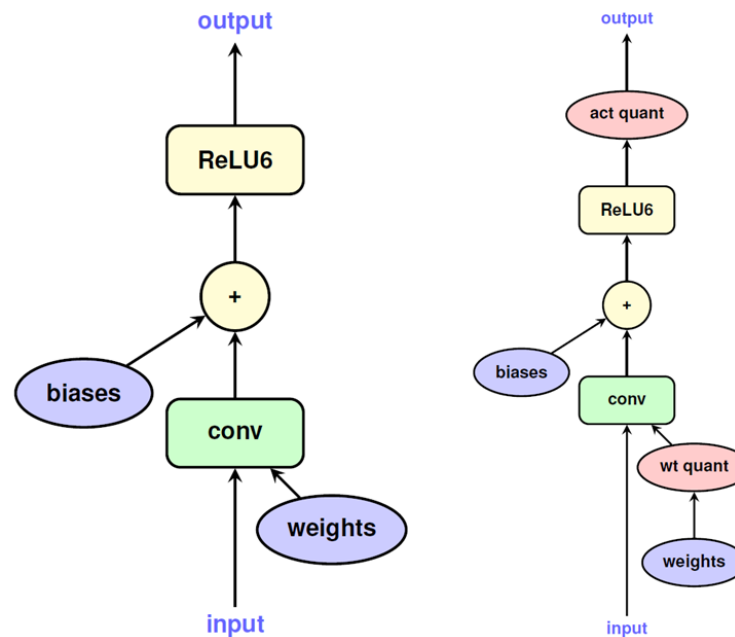
- Use **quantization only in the forward pass**, and replace it with a differentiable function (e.g. identity) in the backward pass.



Quantization-Aware Training

In practice, training is typically still performed in float, but adding “**fake quantization**” ops to the model graph

- In the forward pass, these nodes **simulate the effect of quantization**
- In the backward pass, gradients are propagated through fake-quantization nodes with a STE
- Fake quantizations of activations also record the value range for determining S and z



NEMO: deployment-aware tuning

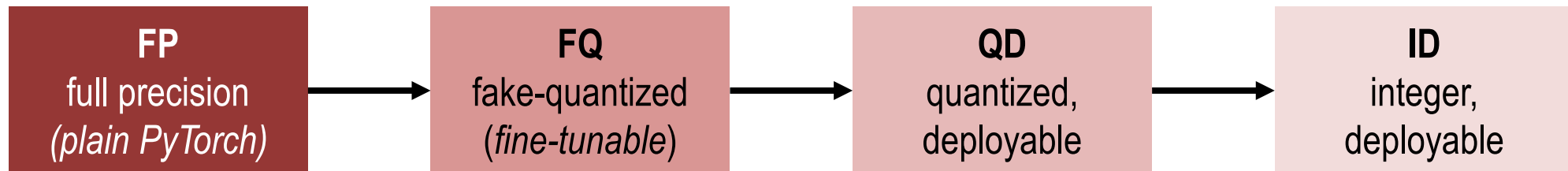


NEMO

NEural Minimization for pyTorch

Key ideas:

- DNN quantization as a **lowering** procedure (like a compiler's IR)



- INT-only hardware results fully reproducible in Python
- implemented as an **imperative library**
 - basically an extension to PyTorch itself, can be integrated in existing PyTorch scripts/flows
 - not black-box: requires domain knowledge (like all ML...)



Pruning



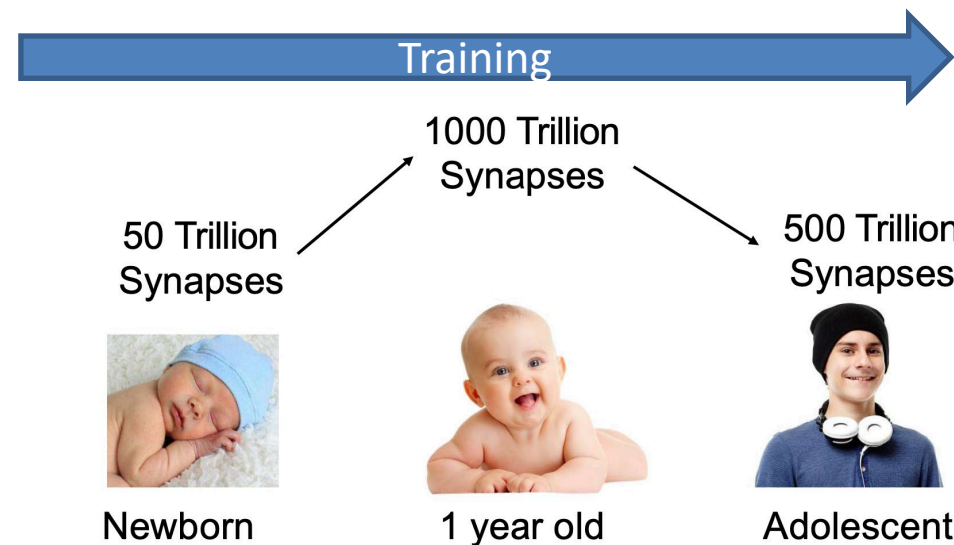
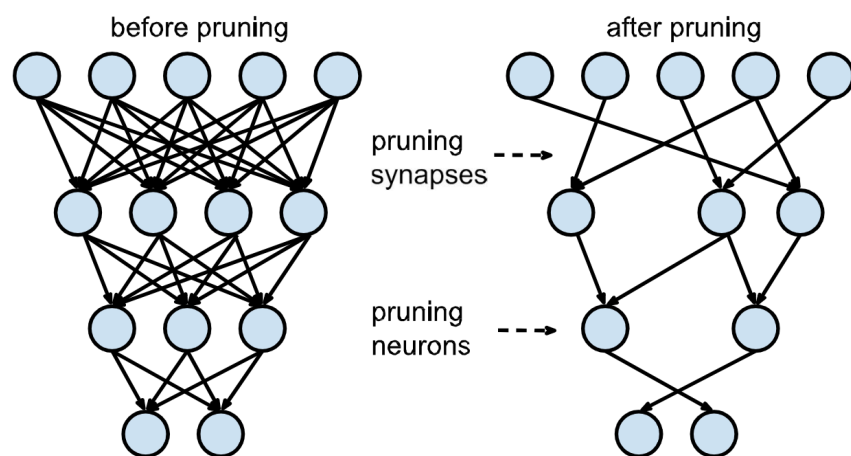
Pruning

Weights pruning: eliminate redundant connections (i.e. weights) in the DNN

- Weights matrices become sparse and can be compressed
- Operations on eliminated weights can be skipped (*)

Activations pruning: Exploit sparsity in activations to skip even more operations

- Secondary optimization, especially on GP hardware



Weights Pruning: Benefits

Extreme model size reductions thanks to **compression**:

- Up to > 90% with no accuracy drop
- Translates into a working-memory reduction only if computations can be performed on compressed sparse formats.

Improved inference speed and energy consumption:

- Only if computations can **actually** be skipped
- Not straight-forward, even on MCUs → Structured pruning.



Weights Pruning: Strategies

Weights pruning is almost invariably enforced at training time.

Different pruning algorithms vary in the objective function used to decide which weights can be eliminated

Two main approaches:

- **Weight Saliency Pruning** (mostly historical)
- **Magnitude-based Pruning** (current standard)



Weights Saliency Pruning

Weights Saliency: impact of eliminating a weight (u) on the objective function (E), i.e. the training loss.

Goal: prune the weights with the minimal impact on E

– Originally proposed in Y. LeCun et al, “Optimal Brain Damage”

E approximated by Taylor expansion:

$$\delta E = \sum_i g_i \delta u_i + \frac{1}{2} \sum_i h_{ii} \delta u_i^2 + \frac{1}{2} \sum_{i \neq j} h_{ij} \delta u_i \delta u_j + O(\|\delta \mathbf{U}\|^3) \quad \text{where} \quad g_i = \frac{\partial E}{\partial u_i} \quad \text{and} \quad h_{ij} = \frac{\partial^2 E}{\partial u_i \partial u_j}$$

Intractable in practice. The Hessian matrix $H = \{h_{ij}\}$ is huge for modern DNNs

Several approximations proposed in the original paper



Magnitude-Based Pruning

Modern approach to weights pruning, based on iteratively eliminating the weights whose **absolute value** (magnitude) **is smaller than a threshold**

- Suitable for large-scale DNNs

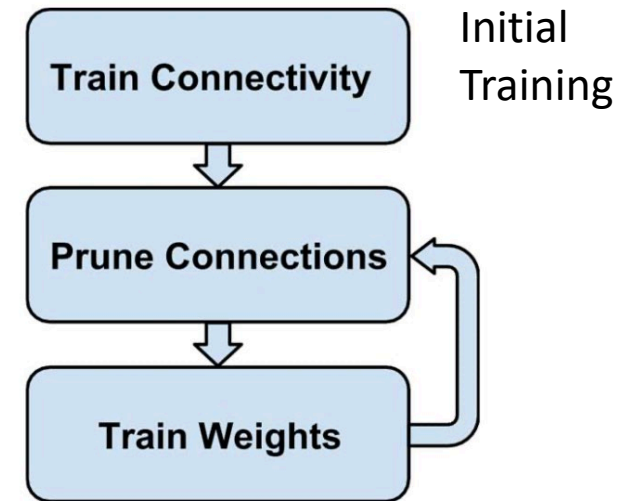
Prune Connections: Eliminate weights with value lower than a threshold

0.4	0.3	3.2	4.1
-3	-2	-0.5	6.3

Th=1

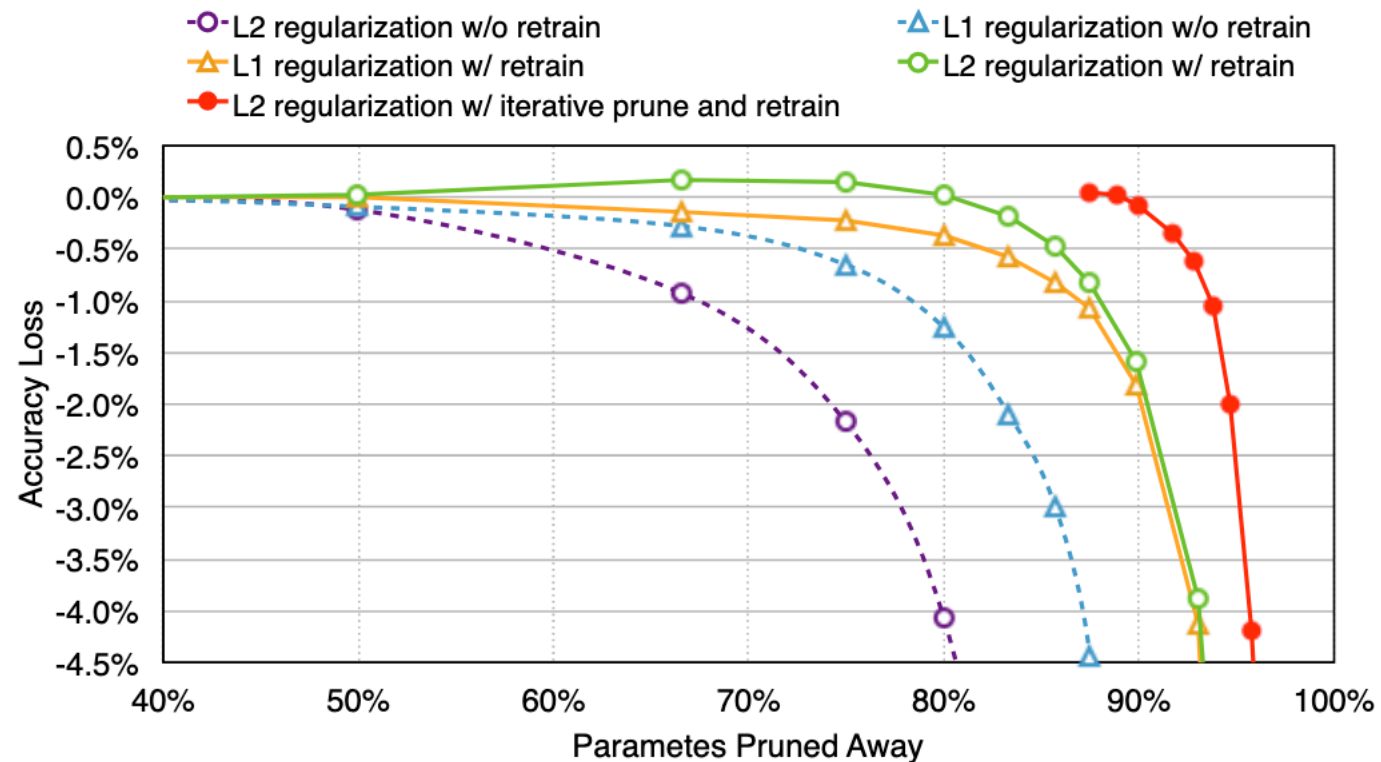
0	0	3.2	4.1
-3	-2	0	6.3

Train Weights: Perform some training steps using masking to make sure that pruned weights are ignored



Magnitude-Based Pruning

Retraining and iterating are fundamental (example on AlexNet)



Han et al. Learning both Weights and Connections for Efficient Neural Networks, NIPS'15

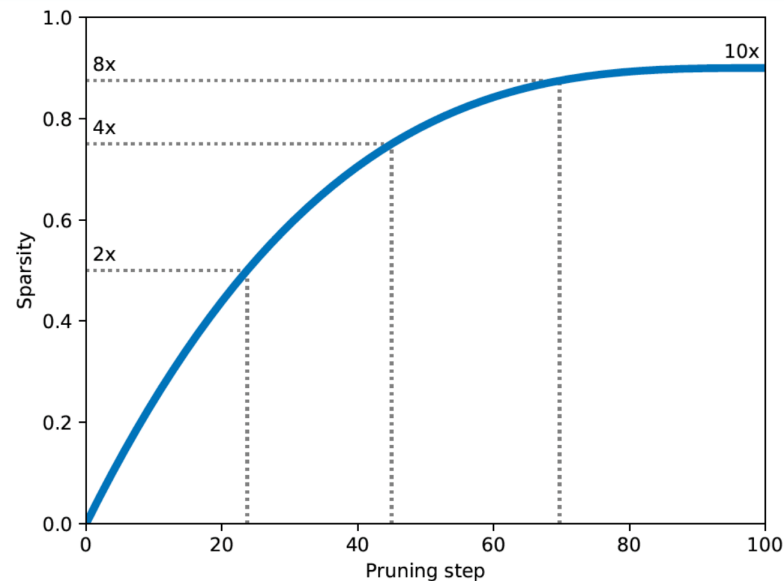


Pruning Schedules

Gradually increasing the amount of pruned weights may yield superior performance compared to “constant” pruning.

Many so-called “pruning schedules” have been proposed

Example: polynomial decay (TFLite)



Pruning Objectives

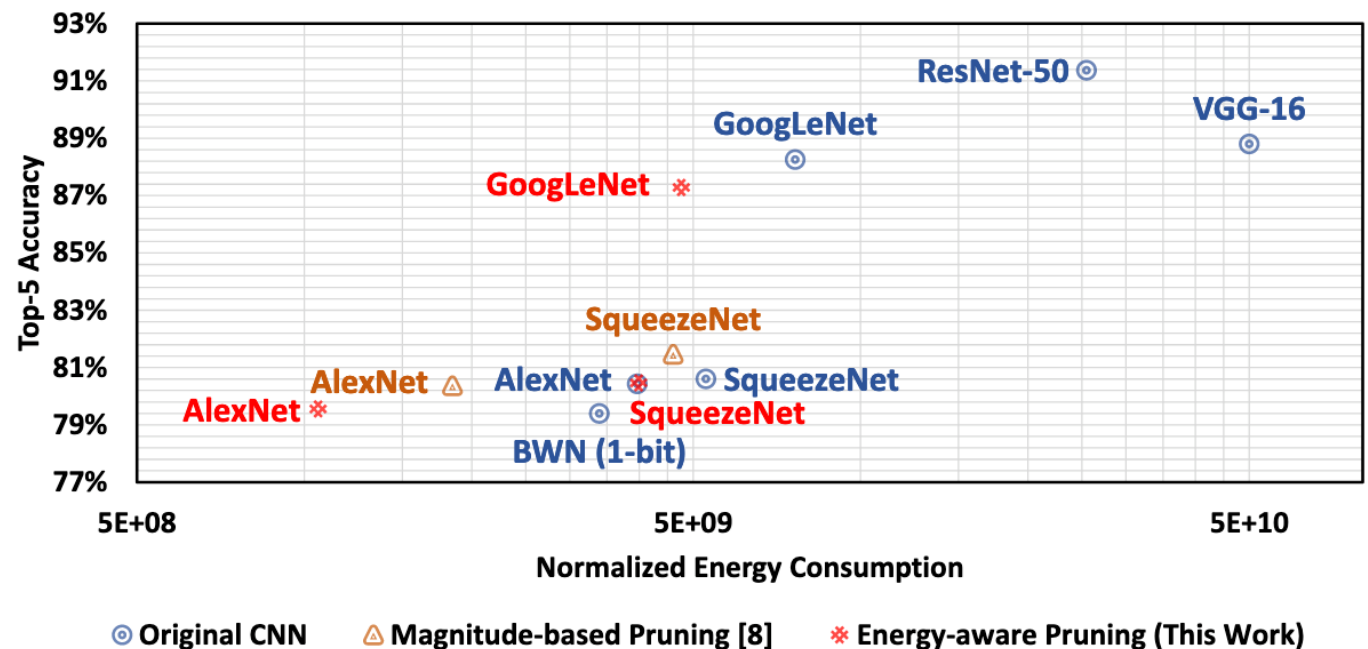
Pure magnitude-based pruning aims at eliminating as many weights as possible

- However, depending on the hardware and objective, **not all layers are equally critical**.

Considering **energy consumption**, a layer's criticality depends on the amount of memory accesses at different levels (hence the data reuse patterns), the number of computations, etc.

Pruning methods can be adapted to these objectives, for instance assigning a priority to different layers.

Energy-aware pruning requires an energy model of the target HW.



Unstructured Pruning Limitations

Limitations of **unstructured** (i.e., purely objective-driven) **pruning**:

- Additional cost for decoding CSR and CSC
- Patterns of non-zero elements are **irregular**

An irregular number of non-zero elements makes it difficult to fully exploit:

- **Caches** (which work better with regular access patterns)
- **Hardware parallelism** (SIMD operations in a CPU, GPU threads, etc).

So, unstructured pruning often makes inference **slower**, or only marginally faster.



Unstructured Pruning Limitations

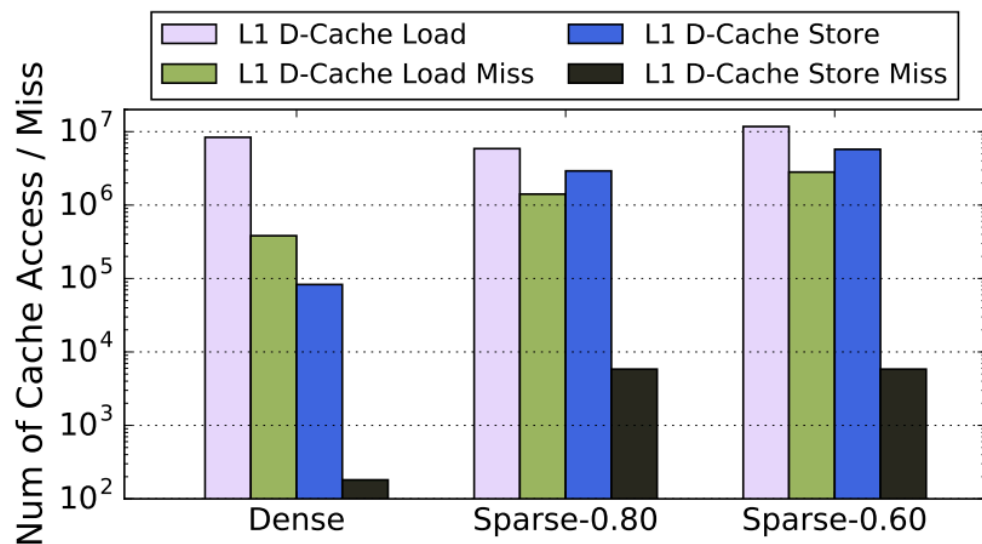


Figure 5: Numbers of cache access and cache miss in the computation of the second CONV layer (*conv2*) in AlexNet on Intel Core i7-6700 CPU. The original dense layer (Dense), the sparse layer with 80% (Sparse-0.80) and 60% (Sparse-0.60) of weights removed are tested. The matrices are randomly generated.

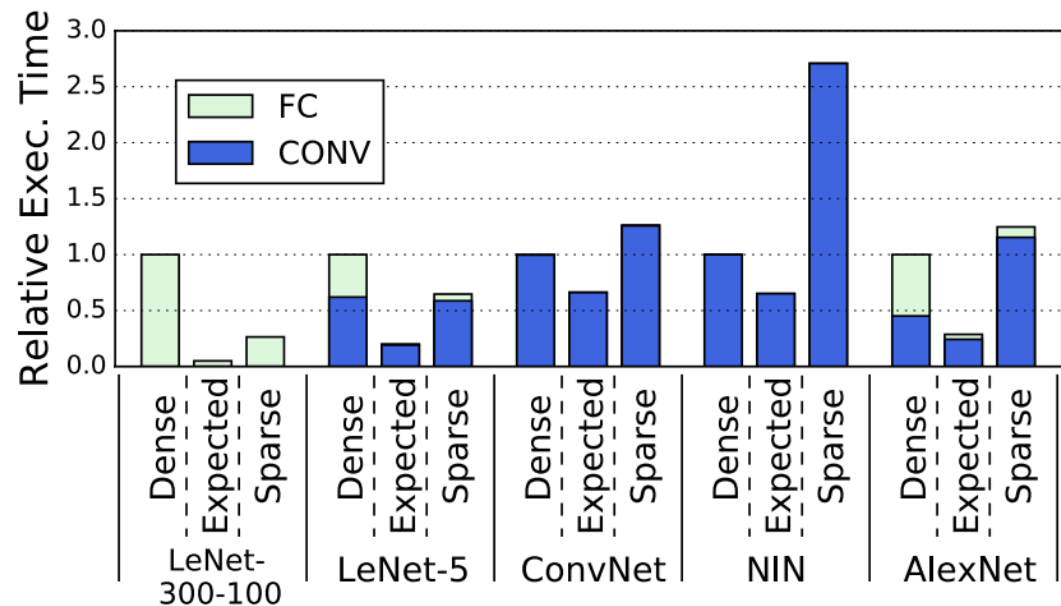


Figure 4: Execution time breakdown of the original networks (Dense) and the pruned networks (Sparse). The expected execution time of pruned networks (Expected) shows the relative MAC operations remaining. The second CONV layer (*conv2*) in AlexNet is tested.

[Source] Yu et al, *Scalpel: Customizing DNN pruning to the underlying hardware parallelism*

Structured Pruning

ASICs can be designed to efficiently handle sparsity

On GP hardware, the most popular solution **structured pruning**

- A constrained pruning process where model compression is balanced with HW-compatibility.

Many variants proposed. We'll overview the two basic ones:

1. Block-based (or SIMD-aware) pruning
2. Node (or channel)-based pruning.



Block-Based Pruning

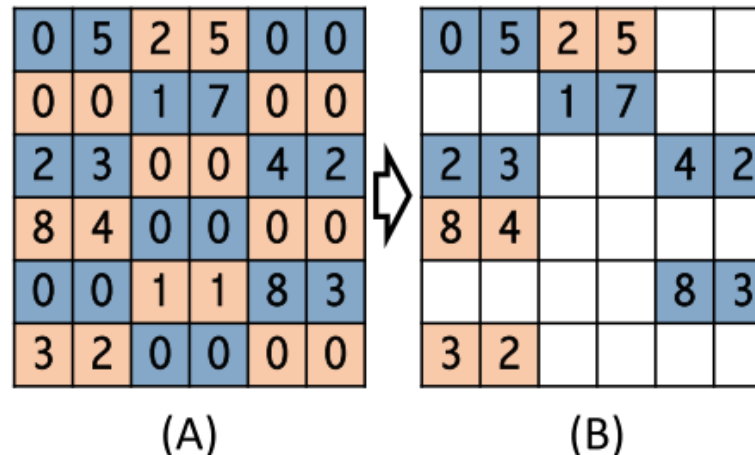
Group together weights based on the underlying SIMD parallelism.

- Enable SIMD loads/stores and MACs

Prune entire groups of weights, based on an aggregated metric

- e.g. Root Mean Square value

Effective when the important goal is to **maximize HW utilization**, and caches are not the main source of sub-optimality (e.g. MCUs)

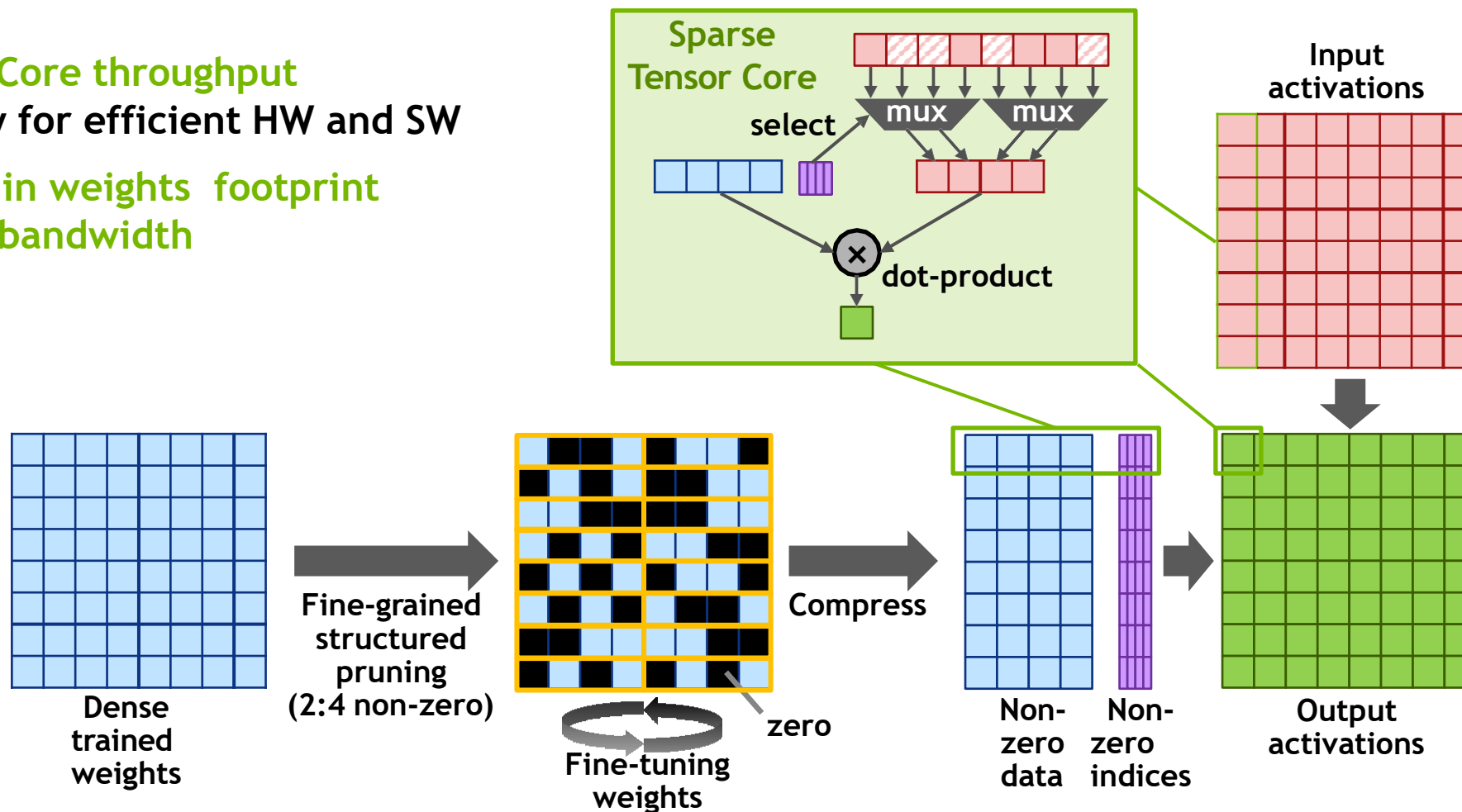


[Source] Yu et al, *Scalpel: Customizing DNN pruning to the underlying hardware parallelism*



NVIDIA Ampere A100 Sparse Tensor Core

2x Tensor Core throughput
Structured-sparsity for efficient HW and SW
~2x reduction in weights footprint
and bandwidth



~No loss in inferencing accuracy

Evaluated across dozens of networks: vision, object detection, segmentation,
natural language modeling, translation

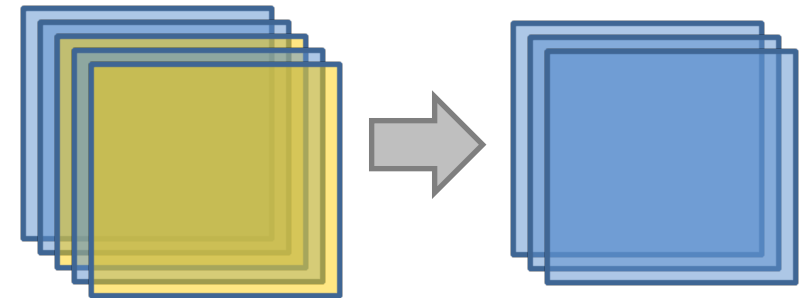
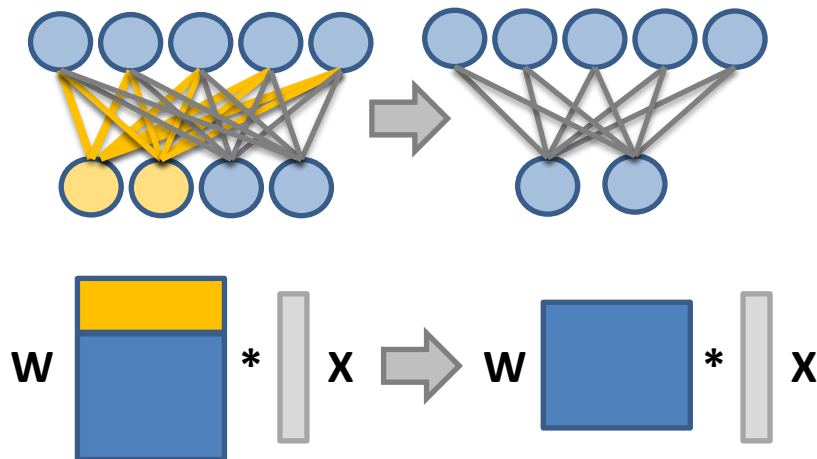
Node-Based Pruning

Eliminate **entire nodes** instead of individual weights (connections).

- Entire “neurons”/units in a FC layer
- Entire feature maps in a Conv layer

Rather than making the layer sparse, it just **shrinks it....**

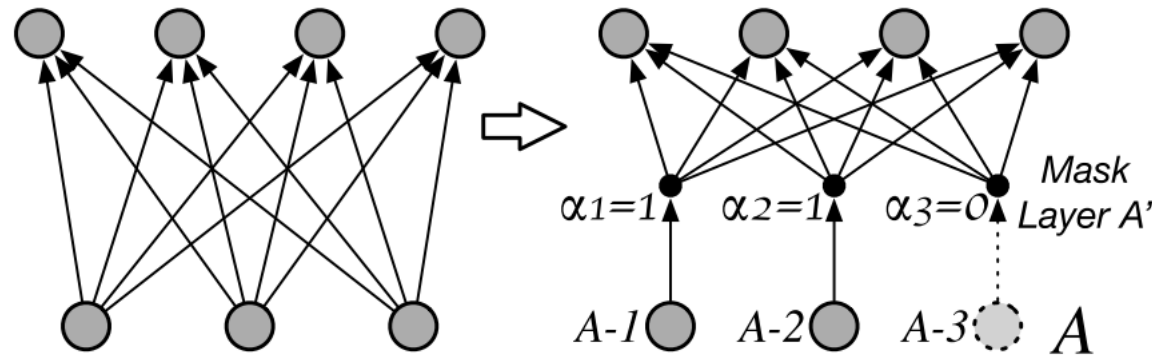
- The final model is equivalent to a DNN with a smaller number of neurons/channels
- Entire “rows” removed from the weights matrix, which remains dense.
- Memory and compute benefits become “obvious”.



Node-Based Pruning

One way to obtain node-based pruning is with “mask” layers:

- Added in-between normal layers
- Multiply their input by a boolean, i.e. $\{0, 1\}$, weight α_i .
- In Conv layers, all elements in the same channel are multiplied by the same α_i



[Source] Yu et al, *Scalpel: Customizing DNN pruning to the underlying hardware parallelism*

Node-Based Pruning

Train the α_i together with the model, forcing them to zero for the least important nodes/channels.

For each α_i , a floating point “**shadow-copy**” β_i is maintained during training and updated with back-propagation (similar to quantization-aware training).

The conversion uses a simple thresholding (in practice, with hysteresis...).

$$\alpha_i = \begin{cases} 1 & \text{if } \beta_i > T \\ 0 & \text{otherwise} \end{cases}$$



Node-Based Pruning

Weight decay regularization is used to force β_i to zero during training.

$$L' = L(y, \hat{y}) + \lambda * \sum |\beta_i|$$

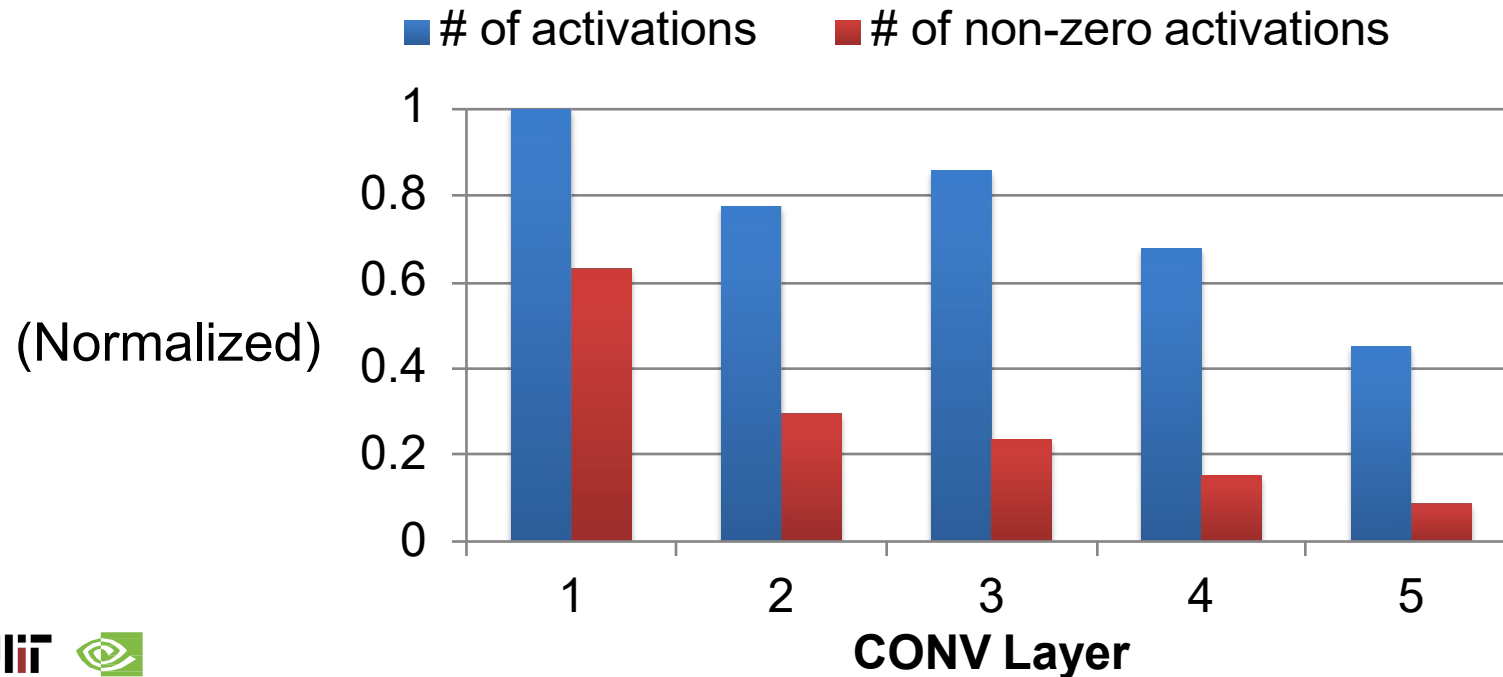
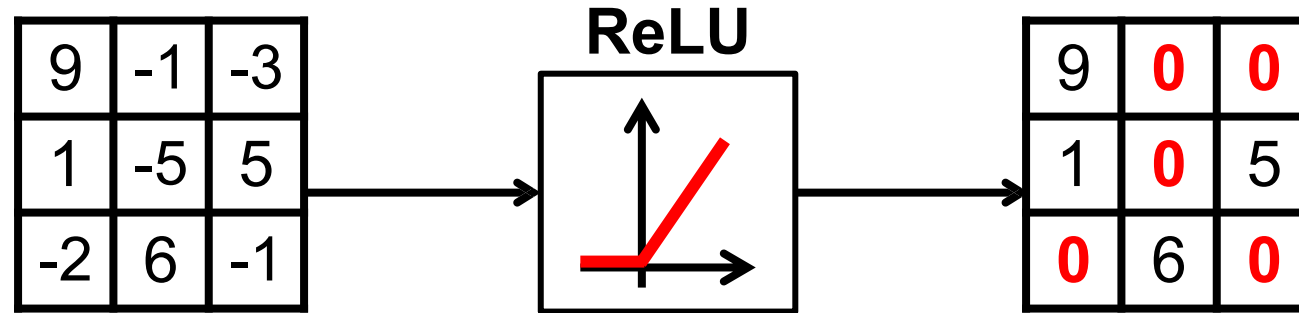
- The parameter λ is increased progressively during training
- Finally, mask layers are removed, and units/channel with $\alpha_i = 0$ are eliminated

Can be seen as a form of **Neural Architecture Search** (NAS).

- Same final complexity and accuracy of a smaller (dense) NN
- But the optimal layers' sizes (nodes/channels) is determined automatically rather than manually by trial and error or grid search.
- Popular lightweight NASs use this concept (e.g., Google's MorphNet).

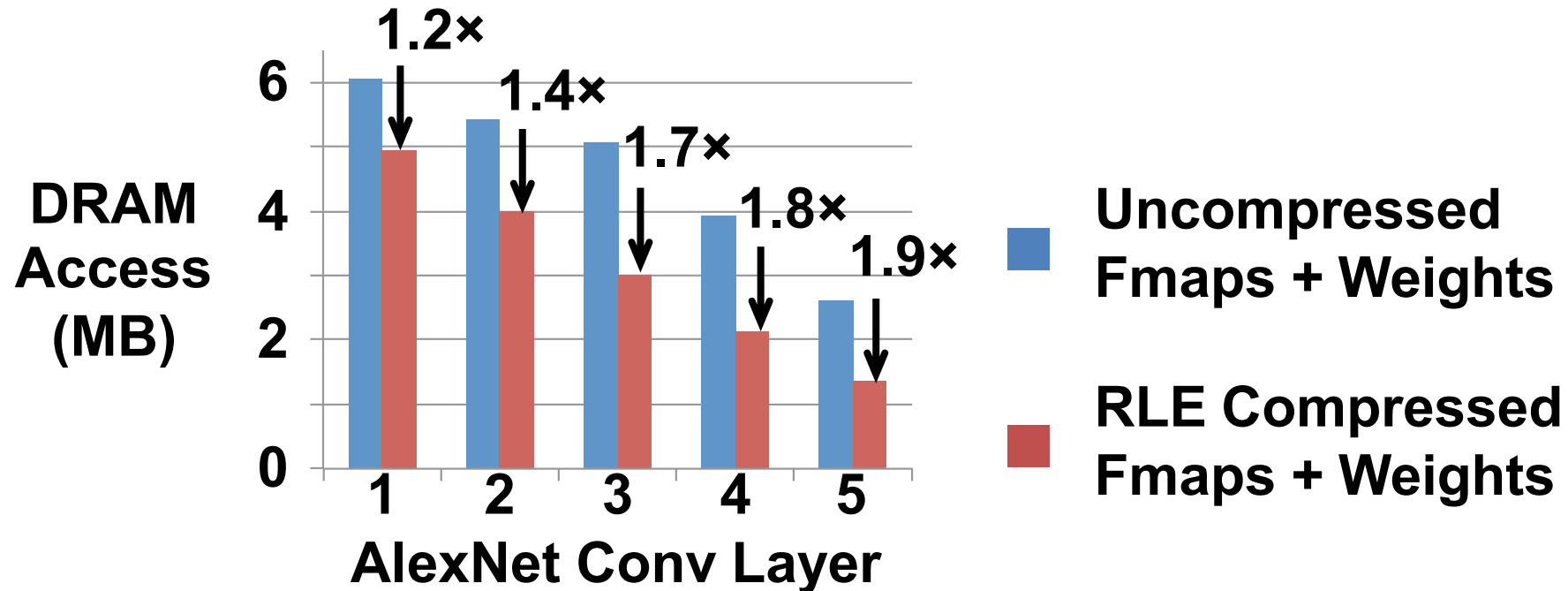


Many **zeros** in **output fmaps** after **ReLU**



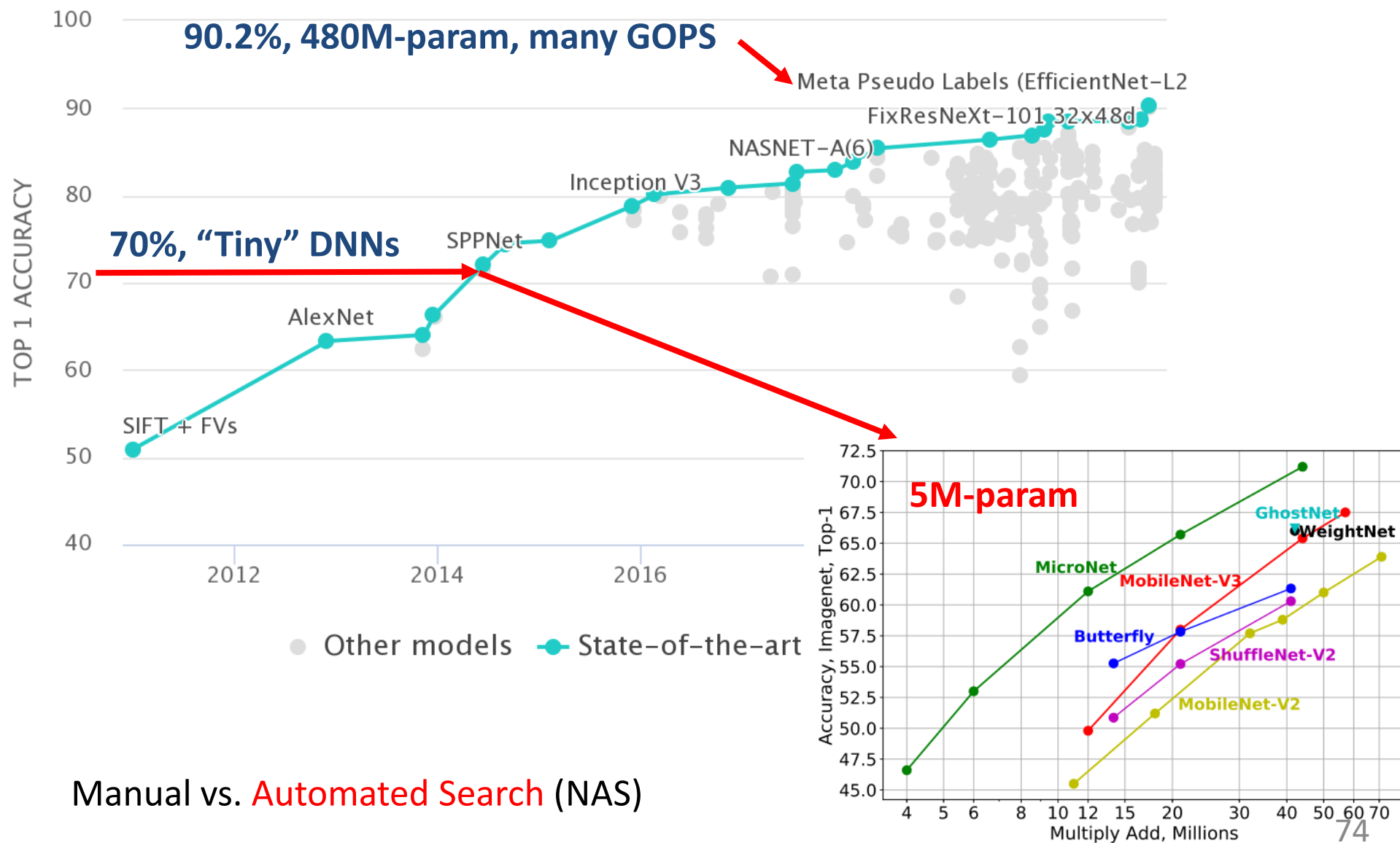
Sparsity → Compression

Run-length compression (sequences of zeros replaced with run-length count)



Simple RLC within 5% - 10% of theoretical entropy limit

Compact Network Architectures



Manual vs. **Automated Search (NAS)**

